



Workshop on Advances in Hardware Security



MaskedHLS: Domain-Specific High-Level Synthesis of Masked Cryptographic Designs

Dr. Chandan Karfa

Indian Insititute of Technology Guwahati

15/02/2025

Outline

1. Background
   - PSCAs
   - Masking
   - Masking Gadgets
   - Hardware vs Software Masking
   - Software Masking
   - Hardware Masking
   - HLS
2. MaskedHLS: domain-specific HLS
   - Impact of HLS
   - Domain-Specific HLS for Masked Designs
   - MaskedHLS Flow
3. Results
   - Results and Conclusion

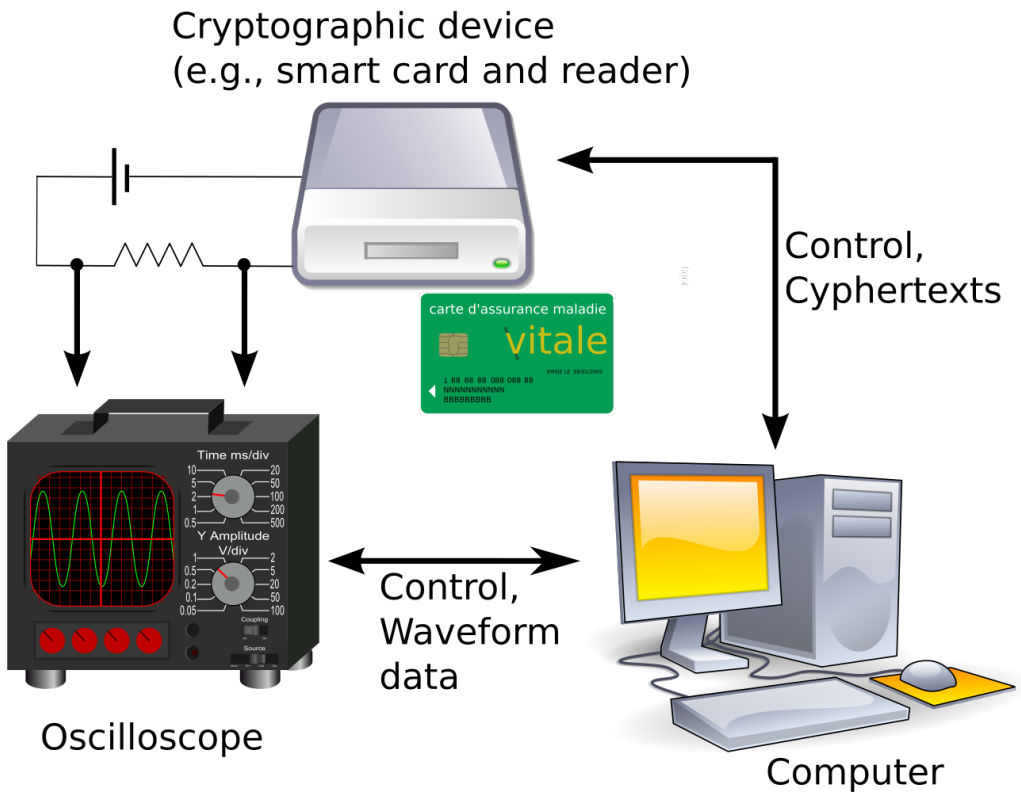
Outline

1 Background
- PSCAs
- Masking
- Masking Gadgets
- Hardware vs Software Masking
- Software Masking
- Hardware Masking
- HLS

2 MaskedHLS: domain-specific HLS
- Impact of HLS
- Domain-Specific HLS for Masked Designs
- MaskedHLS Flow

3 Results
- Results and Conclusion

Power Side-Channel Attacks



Kocher, P. "Differential power analysis." Proc. Advances in Cryptology (CRYPTO'99). 1999.

Power Side-Channel Attacks (PSCAs)

- **Definition**: Exploitation of power consumption information to deduce secret values.
- **Countermeasure**: Masking (splitting secret inputs into independent random shares).
- **Challenges in Hardware**: Glitches can reveal correlations between power consumption and secret data.

Masking

Intuition is to break the correlation between secret(s) and power consumption by randomizing the intermediate computations.

- Let the secret x be splitted into $d+1$ shares, denoted as $x_0, x_1, \ldots, x_d$, such that:

$$x = x_0 \oplus x_1 \oplus \cdots \oplus x_d$$

- Each share x_i (for $i = 0, 1, \ldots, d$) is uniformly random.
- To ensure security, at most d shares can be combined without revealing any information about x.

Chari, Suresh, et al. "Towards sound approaches to counteract power-analysis attacks." Advances in CryptologyCRYPTO99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 1519, 1999 Proceedings 19. Springer Berlin Heidelberg, 1999.

Masking

Sharing of x (Secret Input)

$x \oplus r_1, r_2, r_1 \oplus r_2$ *such that* $x = (x \oplus r_1) \oplus (r_2) \oplus (r_1 \oplus r_2)$

$((x \oplus r_1) \oplus r_2)$, $((r_1 \oplus r_2) \oplus r_2)$, $((x \oplus r_1) \oplus (r_1 \oplus r_2))$ do not leak x
Only a combination of all shares leak x via
$x = (x \oplus r_1) \oplus (r_2) \oplus (r_1 \oplus r_2)$

Objective

Making sure that not all (3) shares of a secret inputs are combined at a single computation.

Masking

Sharing of x (Secret Input)

$x \oplus r_1, r_2, r_1 \oplus r_2$ *such that* $x = (x \oplus r_1) \oplus (r_2) \oplus (r_1 \oplus r_2)$

Objective

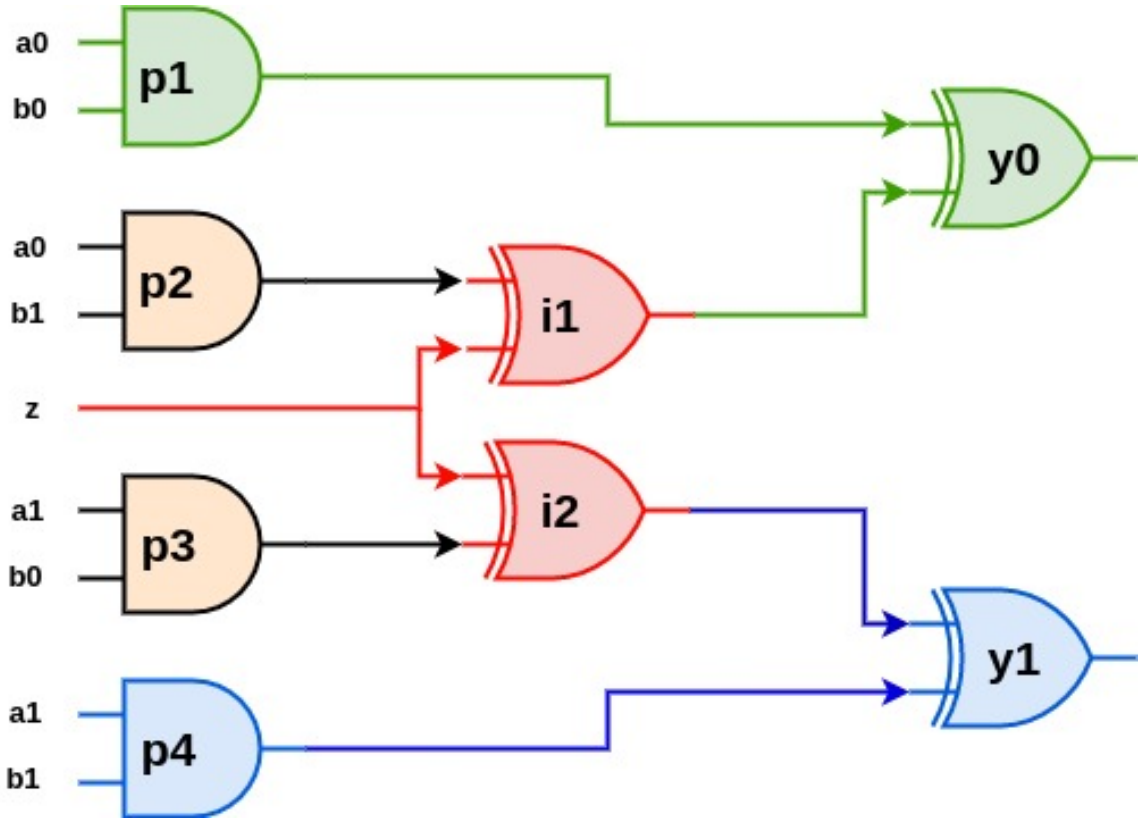
Making sure that not all (3) shares of a secret inputs are combined at a single computation.

Rewrite Computations

Redefine all computations in terms of shares.

Unsplittable computations: Non-linear operations that combine different shares

A (bitwise) multiplication operation having product terms that need to be combined to ensure **correctness**



How to split the unsplittable: Masking Gadgets

- **Gadgets**:
  - **DOMAND**: Domain-oriented masking gadget.
  - **HPC1 & HPC2**: High-performance glitch-resistant gadgets.
  - **COMAR**: Advanced masking gadget, offering higher security.
- **Role**: Each gadget requires specific register placement to avoid leakage.

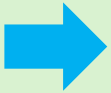
Hardware vs Software Masking

Hardware masking

Software masking

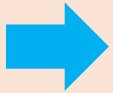


Rewrite the implementation in terms of shares





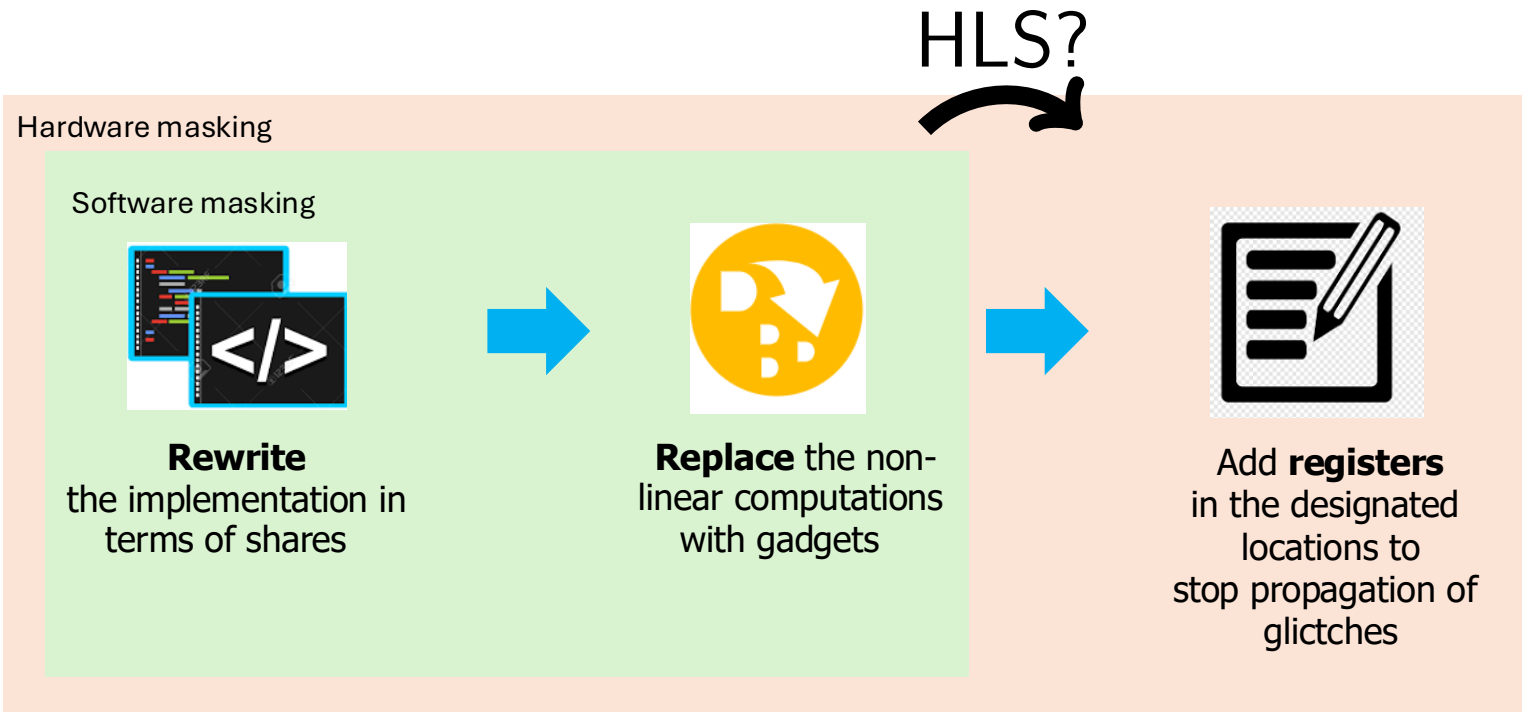
Replace the non-linear computations with gadgets



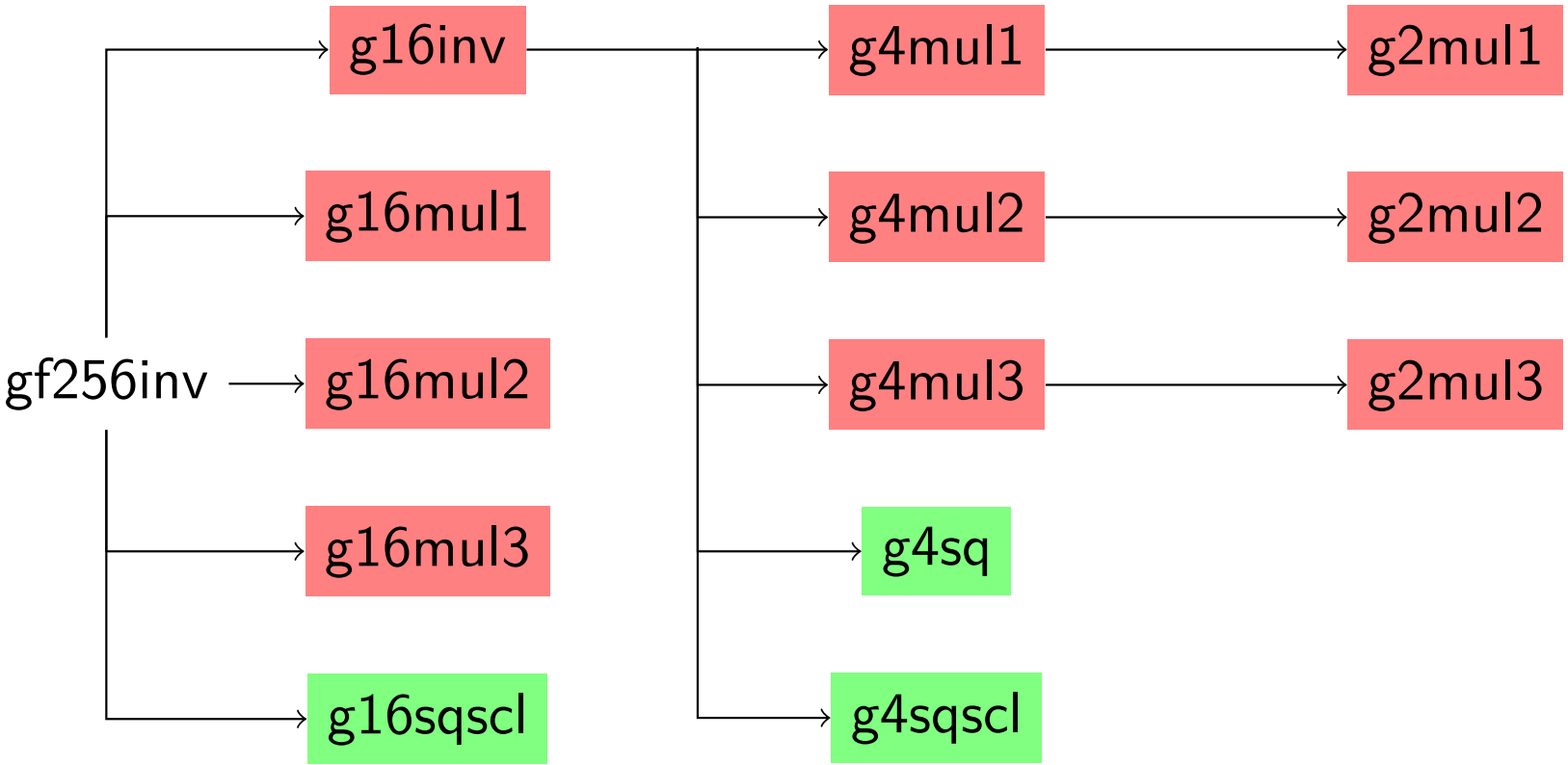


Add **registers** in the designated locations to stop propagation of glictches

HLS?

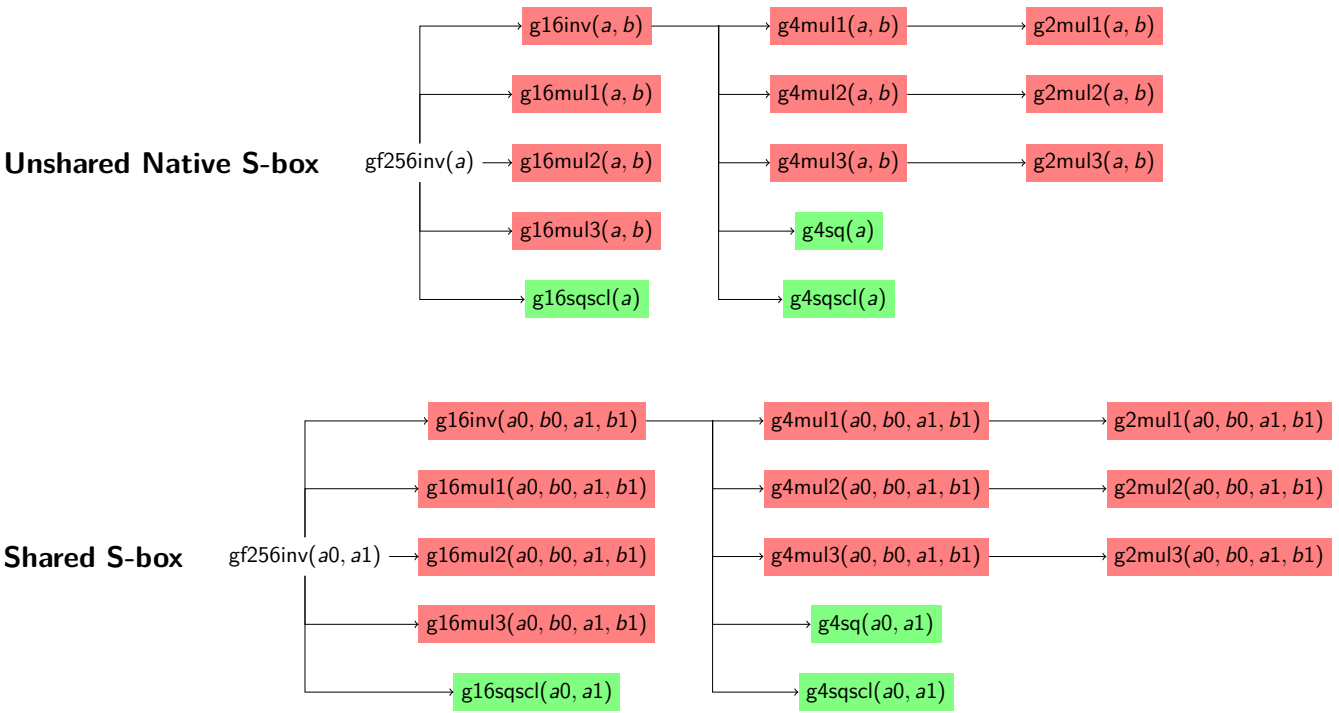

HLS?
Hardware masking
Software masking
Rewrite
the implementation in
terms of shares
Replace the non-
linear computations
with gadgets
Add registers
in the designated
locations to
stop propagation of
glictches

Masking the AES-256 Canright's S-box



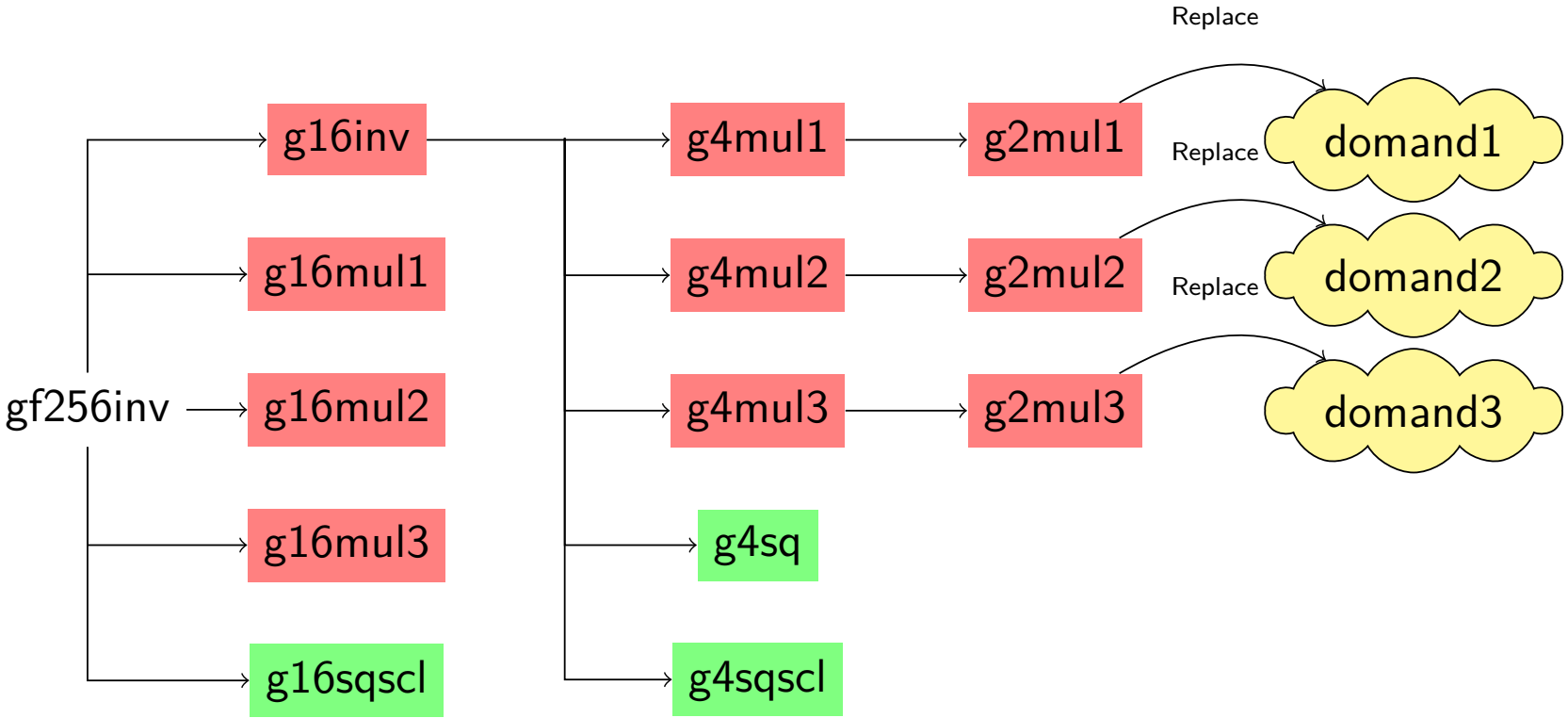
Masking the AES-256 Canright's S-box

Redefining computation in terms of shares: sharing functions.



Masking the AES-256 Canright's S-box

Redefining computations in terms of shares: Replacing non-linear ops with gadgets.



Masking the AES-256 Canright's S-box

```
ap_int<8> G256_inv(ap_int<8> x) {
    ap_int<8> a, b, c, d, e, p, q;
    a = (x & 0xF0) >> 4;
    b = (x & 0x0F);
    c = G16_sq_scl(a ^ b);
    d = G16_mul(a, b);
    e = G16_inv(c ^ d);
    p = G16_mul(e, b);
    q = G16_mul(e, a);
    return ((p << 4) | q);
}
```

```
ap_int<8> G16_inv(ap_int<8> x) {
    ap_int<8> a, b, c, d, e, p, q;
    a = (x & 0xC) >> 2;
    b = (x & 0x3);
    c = G4_scl_N(G4_sq(a ^ b));
    d = G4_mul(a, b);
    e = G4_sq(c ^ d);
    p = G4_mul(e, b);
    q = G4_mul(e, a);
    return ((p << 2) | q);
}
```

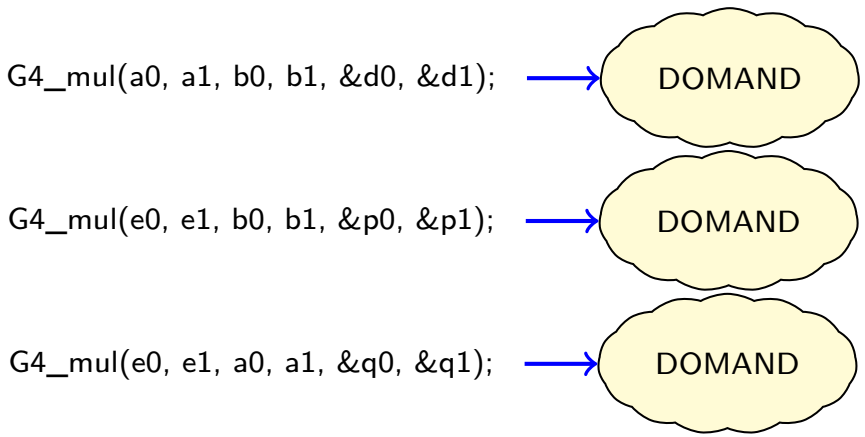
Masking the AES-256 Canright's S-box

```
ap_int<8> G16_inv(ap_int<8> x)
{
    ap_int<8> a, b, c, d,
    e, p, q;
    a_0 = (x & 0xC);
    a = a_0 >> 2;
    b = (x & 0x3);
    c = G4_scl_N(G4_sq(a ^ b))
       ;
    d = G4_mul(a, b);
    e = G4_sq(c ^ d);
    p = G4_mul(e, b);
    q = G4_mul(e, a);
    return ((p << 2) | q);
}
```

```
void G16_inv(ap_int<8> x0, ap_int<8> x1, ap_int<8> *
      y0, ap_int<8> *y1, ){
    ap_int<8> a0, a1, a0_0, a1_0, b0, b1;
    ap_int<8> c0, c1, c0_0, c1_0, d0, d1, e0, e1, p0,
       p1, q0, q1;
    ap_int<8> a0xorb0, a1xorb1, c0xord0, c1xord1;
    a0_0 = x0 & 0xC;    a1_0 = x1 & 0xC;
    a0 = a0_0 >> 2;     a1 = a1_0 >> 2;
    b0 = x0 & 0x3;      b1 = x1 & 0x3;
    a0xorb0 = a0 ^ b0;
    a1xorb1 = a1 ^ b1;
    G4_sq(a0xorb0, a1xorb1, &c0_0, &c1_0);
    G4_scl_N(c0_0, c1_0, &c0, &c1);
    G4_mul(a0, a1, b0, b1, &d0, &d1);
    c0xord0 = c0 ^ d0;
    c1xord1 = c1 ^ d1;
    G4_sq(c0xord0, c1xord1, &e0, &e1);
    G4_mul(e0, e1, b0, b1, &p0, &p1);
    G4_mul(e0, e1, a0, a1, &q0, &q1);
    ap_int<8> p0ls2, p1ls2;
    p0ls2 = (p0 << 2);
    p1ls2 = (p1 << 2);
    *y0 = (p0ls2 | q0);
    *y1 = (p1ls2 | q1);}
```

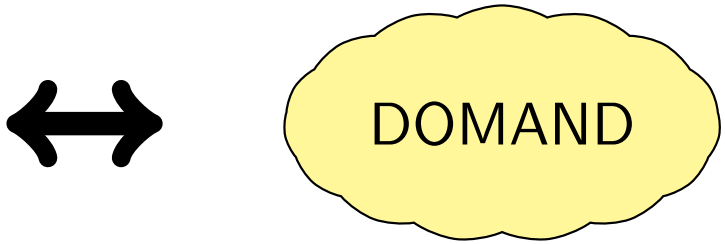
Masking the AES-256 Canright's S-box

```
void G16_inv(ap_int<8> x0, ap_int<8> x1,
ap_int<8> *y0, ap_int<8> *y1, ){
    ap_int<8> a0, a1, a0_0, a1_0, b0, b1;
    ap_int<8> c0, c1, c0_0, c1_0, d0, d1,
      e0, e1, p0, p1, q0, q1;
    ap_int<8> a0xorb0, a1xorb1, c0xord0,
      c1xord1;
    a0_0 = x0 & 0xC;     a1_0 = x1 & 0xC;
    a0 = a0_0 >> 2;      a1 = a1_0 >> 2;
    b0 = x0 & 0x3;       b1 = x1 & 0x3;
    a0xorb0 = a0 ^ b0;
    a1xorb1 = a1 ^ b1;
    G4_sq(a0xorb0, a1xorb1, &c0_0, &c1_0);
    G4_scl_N(c0_0, c1_0, &c0, &c1);
    G4_mul(a0, a1, b0, b1, &d0, &d1);
    c0xord0 = c0 ^ d0;
    c1xord1 = c1 ^ d1;
    G4_sq(c0xord0, c1xord1, &e0, &e1);
    G4_mul(e0, e1, b0, b1, &p0, &p1);
    G4_mul(e0, e1, a0, a1, &q0, &q1);
    ap_int<8> p0ls2, p1ls2;
    p0ls2 = (p0 << 2);
    p1ls2 = (p1 << 2);
    *y0 = (p0ls2 | q0);
    *y1 = (p1ls2 | q1);
 }
```



The DOMAND gadget

```
void domand(bool a0, bool  a1, bool  b0,
bool  b1, bool  *y0, bool  *y1, bool z0){
  int i1, i2, p2, p3, p1, p4;
  p1 = a0 & b0;
  p2 = a0 & b1;
  p3 = a1 & b0;
  p4 = a1 & b1;
  i1 = p2 ^ z0;
  i2 = p3 ^ z0;
  *y0 = i1 ^ p1;
  *y1 = i2 ^ p4;
}
```



GroSS, Hannes, Stefan Mangard, and Thomas Korak. "Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order." Cryptology ePrint Archive (2016).

Masking the AES-256 Canright's S-box

```
void G16_inv(ap_int<8> x0, ap_int<8> x1,
ap_int<8> *y0, ap_int<8> *y1, ){
    ap_int<8> a0, a1, a0_0, a1_0, b0, b1;
    ap_int<8> c0, c1, c0_0, c1_0, d0, d1,
      e0, e1, p0, p1, q0, q1;
    ap_int<8> a0xorb0, a1xorb1, c0xord0,
      c1xord1;
    a0_0 = x0 & 0xC;     a1_0 = x1 & 0xC;
    a0 = a0_0 >> 2;      a1 = a1_0 >> 2;
    b0 = x0 & 0x3;       b1 = x1 & 0x3;
    a0xorb0 = a0 ^ b0;
    a1xorb1 = a1 ^ b1;
    G4_sq(a0xorb0, a1xorb1, &c0_0, &c1_0);
    G4_scl_N(c0_0, c1_0, &c0, &c1);
    G4_mul(a0, a1, b0, b1, &d0, &d1);
    c0xord0 = c0 ^ d0;
    c1xord1 = c1 ^ d1;
    G4_sq(c0xord0, c1xord1, &e0, &e1);
    G4_mul(e0, e1, b0, b1, &p0, &p1);
    G4_mul(e0, e1, a0, a1, &q0, &q1);
    ap_int<8> p0ls2, p1ls2;
    p0ls2 = (p0 << 2);
    p1ls2 = (p1 << 2);
    *y0 = (p0ls2 | q0);
    *y1 = (p1ls2 | q1);
 }
```

Part of g16inv in overall AES S-box inlined software code

```
 .............
d0_G4_mul3_G16_inv0_G256_inv0 = b0_G16_inv0_G256_inv0 & reg(dec_1_inp);
  d1_G4_mul3_G16_inv0_G256_inv0 = b1_G16_inv0_G256_inv0 & reg(dec_1_inp);
  axorb_0_G4_mul3_G16_inv0_G256_inv0 = a0_G4_mul3_G16_inv0_G256_inv0 ^ b0_G4_mul3_G16_inv0_G256_inv0;
  cxord_0_G4_mul3_G16_inv0_G256_inv0 = c0_G4_mul3_G16_inv0_G256_inv0 ^ d0_G4_mul3_G16_inv0_G256_inv0;
  axorb_1_G4_mul3_G16_inv0_G256_inv0 = a1_G4_mul3_G16_inv0_G256_inv0 ^ b1_G4_mul3_G16_inv0_G256_inv0;
  cxord_1_G4_mul3_G16_inv0_G256_inv0 = c1_G4_mul3_G16_inv0_G256_inv0 ^ d1_G4_mul3_G16_inv0_G256_inv0;
  z0_domand0_G4_mul3_G16_inv0_G256_inv0 = r00_G4_mul3_G16_inv0_G256_inv0 % dec_2_inp;
  p2_domand0_G4_mul3_G16_inv0_G256_inv0 = axorb_0_G4_mul3_G16_inv0_G256_inv0 &
            cxord_1_G4_mul3_G16_inv0_G256_inv0;
  i1_domand0_G4_mul3_G16_inv0_G256_inv0 = p2_domand0_G4_mul3_G16_inv0_G256_inv0 ^ (
            z0_domand0_G4_mul3_G16_inv0_G256_inv0);
  p3_domand0_G4_mul3_G16_inv0_G256_inv0 = axorb_1_G4_mul3_G16_inv0_G256_inv0 &
            cxord_0_G4_mul3_G16_inv0_G256_inv0;
  i2_domand0_G4_mul3_G16_inv0_G256_inv0 = p3_domand0_G4_mul3_G16_inv0_G256_inv0 ^ (
            z0_domand0_G4_mul3_G16_inv0_G256_inv0);
  p1_domand0_G4_mul3_G16_inv0_G256_inv0 = axorb_0_G4_mul3_G16_inv0_G256_inv0 &
            cxord_0_G4_mul3_G16_inv0_G256_inv0;
  p4_domand0_G4_mul3_G16_inv0_G256_inv0 = axorb_1_G4_mul3_G16_inv0_G256_inv0 &
            cxord_1_G4_mul3_G16_inv0_G256_inv0;
  e0_G4_mul3_G16_inv0_G256_inv0 = (i1_domand0_G4_mul3_G16_inv0_G256_inv0) ^ (
            p1_domand0_G4_mul3_G16_inv0_G256_inv0);
  e1_G4_mul3_G16_inv0_G256_inv0 = (i2_domand0_G4_mul3_G16_inv0_G256_inv0) ^ (
            p4_domand0_G4_mul3_G16_inv0_G256_inv0);
  z0_domand1_G4_mul3_G16_inv0_G256_inv0 = r10_G4_mul3_G16_inv0_G256_inv0 % dec_2_inp;
  p2_domand1_G4_mul3_G16_inv0_G256_inv0 = a0_G4_mul3_G16_inv0_G256_inv0 & c1_G4_mul3_G16_inv0_G256_inv0;
  i1_domand1_G4_mul3_G16_inv0_G256_inv0 = p2_domand1_G4_mul3_G16_inv0_G256_inv0 ^ (
            z0_domand1_G4_mul3_G16_inv0_G256_inv0);
  p3_domand1_G4_mul3_G16_inv0_G256_inv0 = a1_G4_mul3_G16_inv0_G256_inv0 & c0_G4_mul3_G16_inv0_G256_inv0;
  i2_domand1_G4_mul3_G16_inv0_G256_inv0 = p3_domand1_G4_mul3_G16_inv0_G256_inv0 ^ (
            z0_domand1_G4_mul3_G16_inv0_G256_inv0);
  p1_domand1_G4_mul3_G16_inv0_G256_inv0 = a0_G4_mul3_G16_inv0_G256_inv0 & c0_G4_mul3_G16_inv0_G256_inv0;
  p4_domand1_G4_mul3_G16_inv0_G256_inv0 = a1_G4_mul3_G16_inv0_G256_inv0 & c1_G4_mul3_G16_inv0_G256_inv0;
  p0_0_G4_mul3_G16_inv0_G256_inv0 = (i1_domand1_G4_mul3_G16_inv0_G256_inv0) ^ (
            p1_domand1_G4_mul3_G16_inv0_G256_inv0);
  p1_0_G4_mul3_G16_inv0_G256_inv0 = (i2_domand1_G4_mul3_G16_inv0_G256_inv0) ^ (
            p4_domand1_G4_mul3_G16_inv0_G256_inv0);
  p0_G4_mul3_G16_inv0_G256_inv0 = p0_0_G4_mul3_G16_inv0_G256_inv0 ^ e0_G4_mul3_G16_inv0_G256_inv0;
  p1_G4_mul3_G16_inv0_G256_inv0 = p1_0_G4_mul3_G16_inv0_G256_inv0 ^ e1_G4_mul3_G16_inv0_G256_inv0;
  z0_domand2_G4_mul3_G16_inv0_G256_inv0 = r20_G4_mul3_G16_inv0_G256_inv0 % dec_2_inp;
  p2_domand2_G4_mul3_G16_inv0_G256_inv0 = b0_G4_mul3_G16_inv0_G256_inv0 & c1_G4_mul3_G16_inv0_G256_inv0;
  i1_domand2_G4_mul3_G16_inv0_G256_inv0 = p2_domand2_G4_mul3_G16_inv0_G256_inv0 ^ (
            z0_domand2_G4_mul3_G16_inv0_G256_inv0);
  p3_domand2_G4_mul3_G16_inv0_G256_inv0 = b1_G4_mul3_G16_inv0_G256_inv0 & d0_G4_mul3_G16_inv0_G256_inv0;
  i2_domand2_G4_mul3_G16_inv0_G256_inv0 = p3_domand2_G4_mul3_G16_inv0_G256_inv0 ^ (
```

DOMAND instance

DOMAND instance

DOMAND instance

Masking the AES-256 Canright's S-box at Hardware (RTL): Glitches

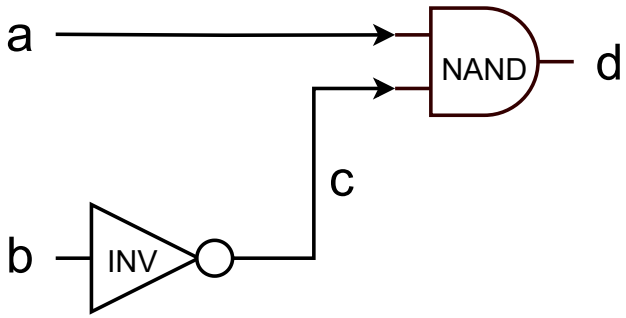


Figure 1: Sample glitchy circuit

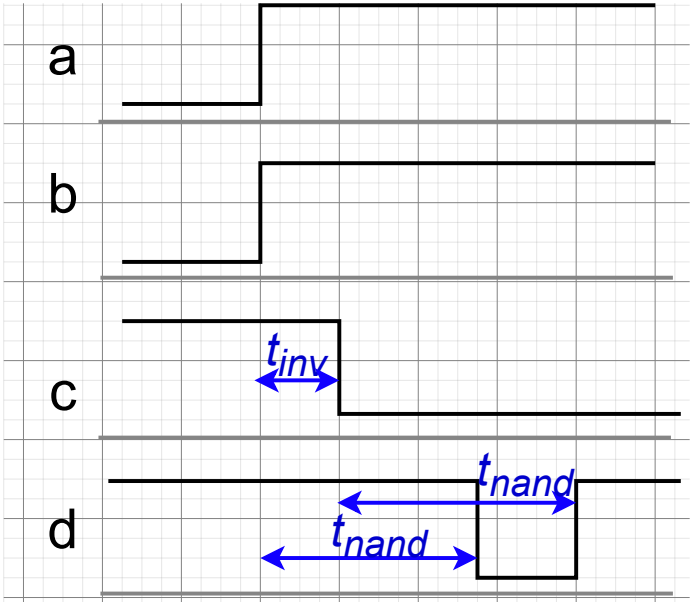
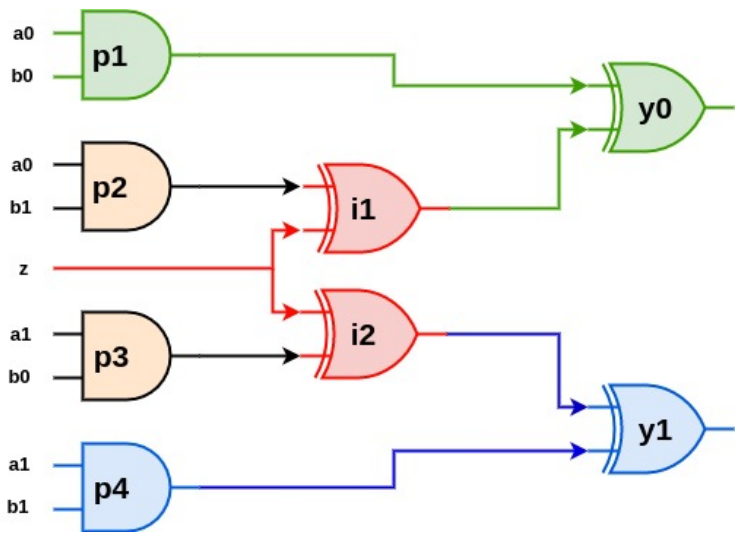


Figure 2: Stabilization of signals in circuit

Glitches affect masking : the solution

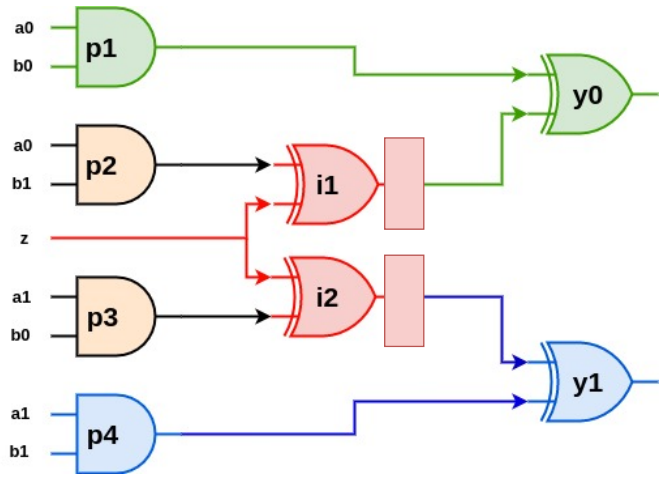
```
void domand(bool a0, bool  a1, bool  b0,
bool  b1, bool  *y0, bool  *y1, bool z0){
  int i1, i2, p2, p3, p1, p4;
  p1 = a0 & b0;
  p2 = a0 & b1;
  p3 = a1 & b0;
  p4 = a1 & b1;
  i1 = p2 ^ z0;
  i2 = p3 ^ z0;
  *y0 = i1 ^ p1;
  *y1 = i2 ^ p4;
}
```



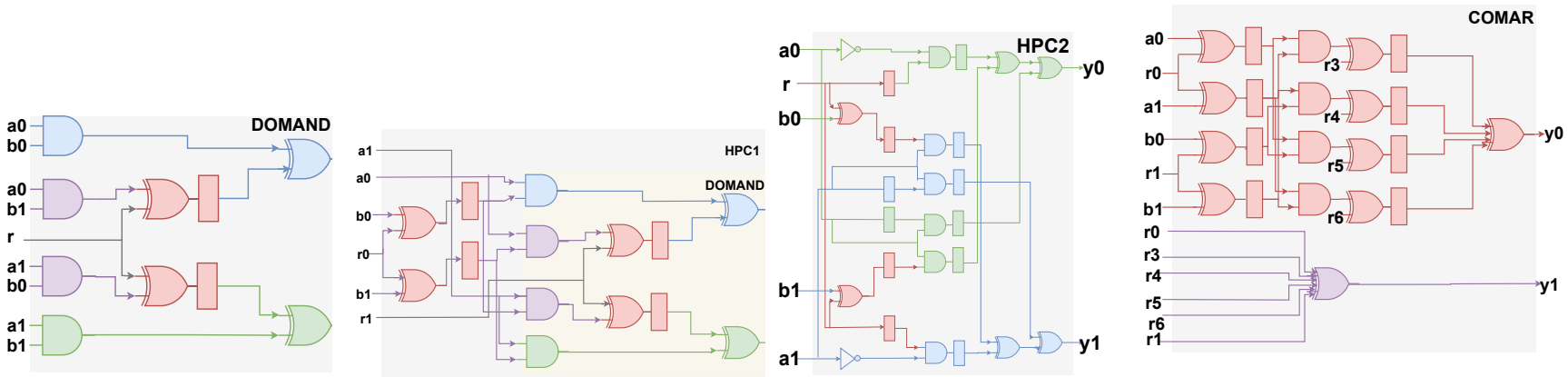
Glitches affect masking : the solution

Add registers at locations deemed sensitive by the Masking Scheme (such as DOM)

```
void domand(bool a0, bool  a1, bool  b0,
bool  b1, bool  *y0, bool  *y1, bool z0){
  int i1, i2, p2, p3, p1, p4;
  p1 = a0 & b0;
  p2 = a0 & b1;
  p3 = a1 & b0;
  p4 = a1 & b1;
  i1 = p2 ^ z0;
  i2 = p3 ^ z0;
  *y0 = reg(i1) ^ p1;
  *y1 = reg(i2) ^ p4;
}
```



Masked Multiplication Gadgets



(a) (b) (c) (d)

Figure 3: Masked Multiplication Gadgets (a) DOMAND (b) HPC1 (c) HPC2 (d) COMAR.

Cassiers, Gaëtan, et al. "Hardware private circuits: From trivial composition to full verification." IEEE Transactions on Computers 70.10 (2020): 1677-1690.
Knichel, David, and Amir Moradi. "Composable Gadgets with Reused Fresh Masks — First-Order Probing-Secure Hardware Circuits with only 6 Fresh Masks." Cryptology ePrint Archive (2023).

Software vs Hardware Masking

Software Masking

```
ap_int<8> G16_inv(ap_int<8> x) {
ap_int<8> a, b, c, d, e, p, q;
  a = (x & 0xC) >> 2;
  b = (x & 0x3);
  c = G4_scl_N(G4_sq(a ^ b));
  d = G4_mul(a, b);
  e = G4_sq(c ^ d);
  p = G4_mul(e, b);
  q = G4_mul(e, a);
   return ((p << 2) | q);
}
```

```
void G16_inv(ap_int<8> x0, ap_int<8> x1,
ap_int<8> *y0, ap_int<8> *y1){
    ap_int<8> a0, a1, a0_0, a1_0, b0, b1;
    ap_int<8> c0, c1, c0_0, c1_0, d0, d1, e0, e1, p0, p1,
q0, q1;
    ap_int<8> a0xorb0, a1xorb1, c0xord0, c1xord1;
    a0_0 = x0 & 0xC;
    a1_0 = x1 & 0xC;
    a0 = a0_0 >> 2;
    a1 = a1_0 >> 2;
    b0 = x0 & 0x3;
    b1 = x1 & 0x3;
    a0xorb0 = a0 ^ b0;
    a1xorb1 = a1 ^ b1;
    G4_sq(a0xorb0, a1xorb1, &c0_0, &c1_0);
    G4_scl_N(c0_0, c1_0, &c0, &c1);

    G4_mul(a0, a1, b0, b1, &d0, &d1);
    c0xord0 = c0 ^ d0;
    c1xord1 = c1 ^ d1;
    G4_sq(c0xord0, c1xord1, &e0, &e1);

    G4_mul(e0, e1, b0, b1, &p0, &p1);
    G4_mul(e0, e1, a0, a1, &q0, &q1);

    ap_int<8> p0ls2, p1ls2;
    p0ls2 = (p0 << 2);
    p1ls2 = (p1 << 2);
    *y0 = (p0ls2 | q0);
    *y1 = (p1ls2 | q1);}
```

Hardware Masking

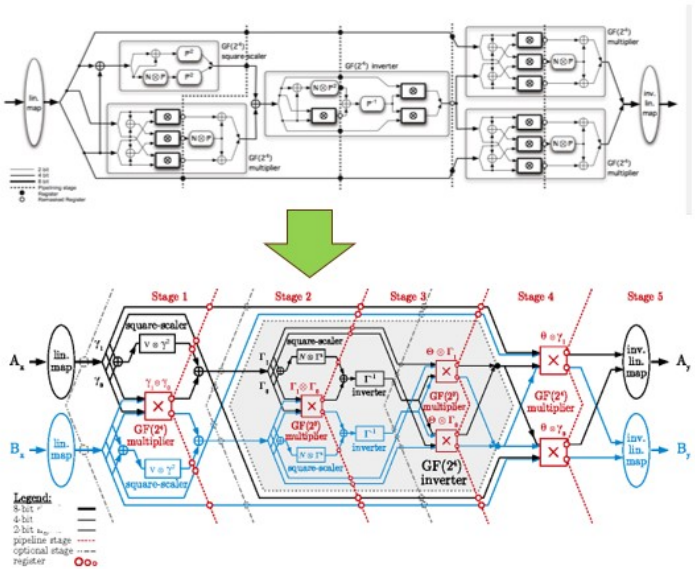
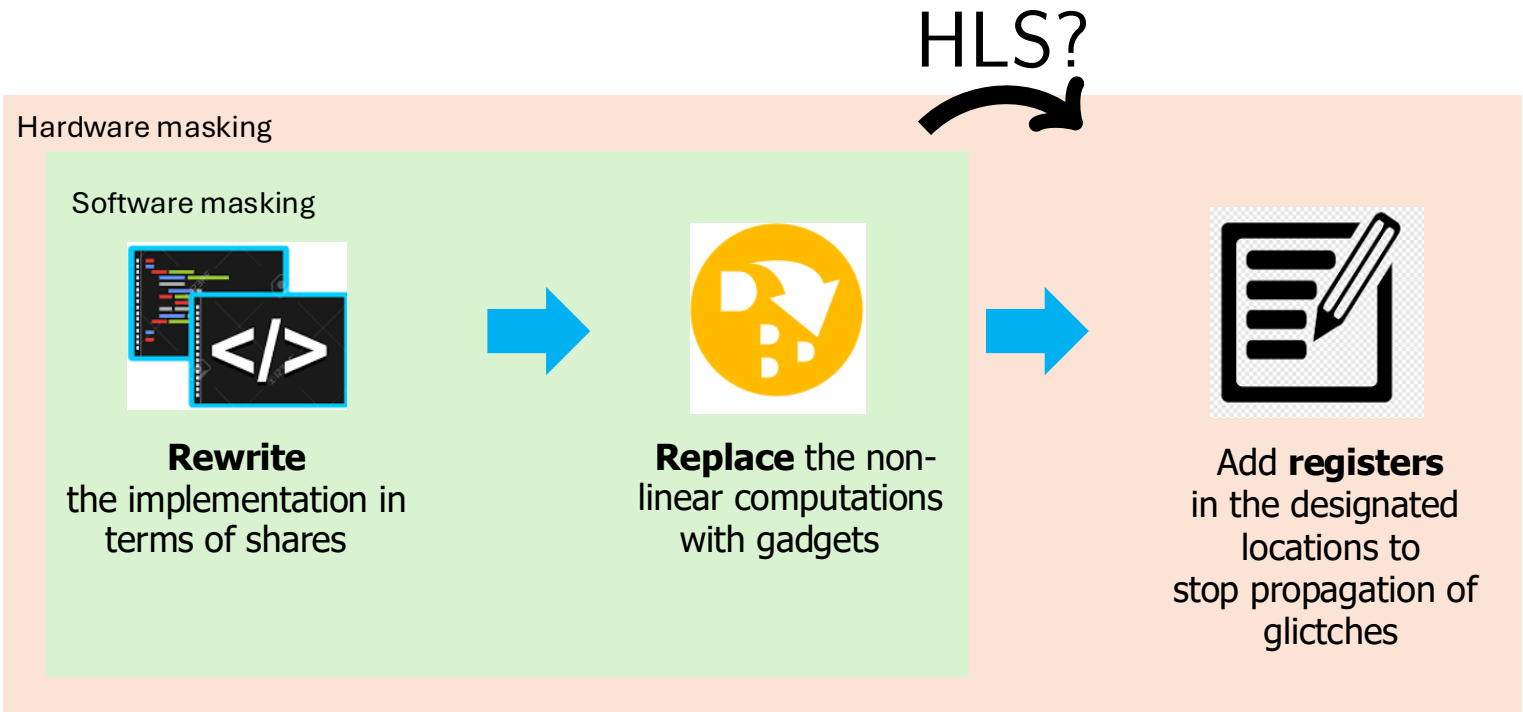


Figure 4: Software vs Hardware Masking

HLS?


HLS?
Hardware masking
Software masking
Rewrite the implementation in terms of shares
Replace the non-linear computations with gadgets
Add registers in the designated locations to stop propagation of glictches

Glitches affect masking : Register insertion via HLS

Add registers at locations deemed sensitive by the Masking Scheme (such as DOM)

```
void domand(bool a0, bool  a1, bool  b0,
bool  b1, bool  *y0, bool  *y1, bool z0){
  int i1, i2, p2, p3, p1, p4;
  p1 = a0 & b0;
  p2 = a0 & b1;
  p3 = a1 & b0;
  p4 = a1 & b1;
  i1 = p2 ^ z0;
  i2 = p3 ^ z0;
  *y0 = reg(i1) ^ p1;
  *y1 = reg(i2) ^ p4;
}
```

```
module domand(
  input a0, a1, b0, b1, z0, clk,
  output reg y0, y1
);
  wire p1, p2, p3, p4;
  reg i1, i2;

  assign p1 = a0 & b0;
  assign p2 = a0 & b1;
  assign p3 = a1 & b0;
  assign p4 = a1 & b1;

  always @(posedge clk) begin
    i1 <= p2 ^ z0;
    i2 <= p3 ^ z0;
    y0 <= i1 ^ p1;
    y1 <= i2 ^ p4;
  end
endmodule
```

Masking the AES S-box: Register insertion via HLS

unmasked g16inv in overall AES S-box in software code

```
 ............
ap_int<8> G16_inv(ap_int<8> x)
{
    ap_int<8> a, b, c, d,
    e, p, q;
    a_0 = (x & 0xC);
    a = a_0 >> 2;
    b = (x & 0x3);
    c = G4_scl_N(G4_sq(a ^ b));
    d = G4_mul(a, b);
    e = G4_sq(c ^ d);
    p = G4_mul(e, b);
    q = G4_mul(e, a);
    return ((p << 2) | q);
}
  .........
```

Part of g16inv in overall AES S-box RTL code

```
............
cxord_1_G4_mul5_G16_inv0_G256_inv0_reg <= cxord_1_G4_mul5_G16_inv0_G256_inv0;
z3811_assgn38110 <= z3811_assgn3811;
z1283_assgn1283 <= z3811_assgn38110;
cxord_0_G4_mul5_G16_inv0_G256_inv0_reg <= cxord_0_G4_mul5_G16_inv0_G256_inv0;
z3817_assgn38170 <= z3817_assgn3817;
z1287_assgn1287 <= z3817_assgn38170;
i1_domand0_G4_mul5_G16_inv0_G256_inv0_reg <= i1_domand0_G4_mul5_G16_inv0_G256_inv0;
p1_domand0_G4_mul5_G16_inv0_G256_inv0_reg <= p1_domand0_G4_mul5_G16_inv0_G256_inv0;
i2_domand0_G4_mul5_G16_inv0_G256_inv0_reg <= i2_domand0_G4_mul5_G16_inv0_G256_inv0;
p4_domand0_G4_mul5_G16_inv0_G256_inv0_reg <= p4_domand0_G4_mul5_G16_inv0_G256_inv0;
c1_G4_mul5_G16_inv0_G256_inv0_reg <= c1_G4_mul5_G16_inv0_G256_inv0;
z3833_assgn38330 <= z3833_assgn3833;
z1301_assgn1301 <= z3833_assgn38330;
c0_G4_mul5_G16_inv0_G256_inv0_reg <= c0_G4_mul5_G16_inv0_G256_inv0;
z3839_assgn38390 <= z3839_assgn3839;
z1305_assgn1305 <= z3839_assgn38390;
i1_domand1_G4_mul5_G16_inv0_G256_inv0_reg <= i1_domand1_G4_mul5_G16_inv0_G256_inv0;
p1_domand1_G4_mul5_G16_inv0_G256_inv0_reg <= p1_domand1_G4_mul5_G16_inv0_G256_inv0;
i2_domand1_G4_mul5_G16_inv0_G256_inv0_reg <= i2_domand1_G4_mul5_G16_inv0_G256_inv0;
p4_domand1_G4_mul5_G16_inv0_G256_inv0_reg <= p4_domand1_G4_mul5_G16_inv0_G256_inv0;
d1_G4_mul5_G16_inv0_G256_inv0_reg <= d1_G4_mul5_G16_inv0_G256_inv0;
z3859_assgn38590 <= z3859_assgn3859;
z1323_assgn1323 <= z3859_assgn38590;
d0_G4_mul5_G16_inv0_G256_inv0_reg <= d0_G4_mul5_G16_inv0_G256_inv0;
z3865_assgn38650 <= z3865_assgn3865;
z1327_assgn1327 <= z3865_assgn38650;
i1_domand2_G4_mul5_G16_inv0_G256_inv0_reg <= i1_domand2_G4_mul5_G16_inv0_G256_inv0;
p1_domand2_G4_mul5_G16_inv0_G256_inv0_reg <= p1_domand2_G4_mul5_G16_inv0_G256_inv0;
i2_domand2_G4_mul5_G16_inv0_G256_inv0_reg <= i2_domand2_G4_mul5_G16_inv0_G256_inv0;
p4_domand2_G4_mul5_G16_inv0_G256_inv0_reg <= p4_domand2_G4_mul5_G16_inv0_G256_inv0;
z3881_assgn38810 <= z3881_assgn3881;
z3881_assgn38811 <= z3881_assgn38810;
z1341_assgn1341 <= z3881_assgn38811;
z3885_assgn38850 <= z3885_assgn3885;
z3885_assgn38851 <= z3885_assgn38850;
  .........
```

registered values

registered values

registered values

HLS?

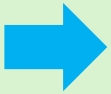
HLS?

Hardware masking

Software masking

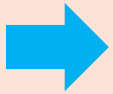


Rewrite
the implementation in terms of shares





Replace the non-linear computations with gadgets





Add **registers** in the designated locations to stop propagation of glictches

Related Work : HLS for Masking

Sadhukhan, Rajat, Sayandeep Saha, and Debdeep Mukhopadhyay. **"Shortest path to secured hardware: Domain oriented masking with high-level-synthesis."** 2021 58th ACM/IEEE Design Automation Conference (DAC). IEEE, 2021.

- Identified that front-end optimizations like reassociation effect masked designed generated by HLS (spc. *Bambu HLS*
- Proposed to switch-off reassociation in *Bambu HLS*

Inagaki, Saya, et al. **"Power Side-channel Attack Resistant Circuit Designs of ARX Ciphers Using High-level Synthesis."** ACM Transactions on Embedded Computing Systems 22.5 (2023): 1-17.

- Identified that operation chained datapath obtained via HLS is not secure against PSCAs.

Problems:

- Both these work propose to deal with **individual optimizations** performed by **specific HLS tools** by switching them off in a **specific set of designs.**
- Impact of **unexplored optimizations** not studied.
- **No guarantee** their solution is generic for other designs
- Ignore the larger security requirement of **inserting registers with optimal latency and balanced parallel paths**.

Outline

1 Background
- PSCAs
- Masking
- Masking Gadgets
- Hardware vs Software Masking
- Software Masking
- Hardware Masking
- HLS

2 MaskedHLS: domain-specific HLS
- Impact of HLS
- Domain-Specific HLS for Masked Designs
- MaskedHLS Flow

3 Results
- Results and Conclusion

Why domain-specific HLS? Impact of HLS on PSCA Security

- **HLS Optimizations**:
  - Can compromise masking security by altering the design.
  - **Front End:** Issues with LLVM/GCC optimizations, such as reassociation, cause insecure recombinations.
  - **Back-End:** Issues with the glitches introduced by the controller and its interaction with the masking security.
  - Standard HLS tools lack guarantees for PSCA security.

Impact of HLS on PSCA Security

Front-End HLS Optimizations

```
void domand(bool a0, bool a1,
      bool b0,
 bool b1, bool *y0, bool *y1,
      bool z0)
 {
  int i1, i2, p2, p3, p1, p4;
  p1 = a0 & b0;
  p2 = a0 & b1;
  p3 = a1 & b0;
  p4 = a1 & b1;
  i1 = p2 ^ z0;
  i2 = p3 ^ z0;
  *y0 = i1 ^ p1;
  *y1 = i2 ^ p4;
}
```

reassociation →

```
void domand(bool a0, bool a1,
      bool b0,
bool b1, bool *y0, bool *y1, bool
       z0)
{
  int i1, i2, p2, p3, p1, p4;
  p1 = a0 & b0;
  p2 = a0 & b1;
  p3 = a1 & b0;
  p4 = a1 & b1;
  i1 = p1 ^ z0;
  i2 = p4 ^ z0;
  *y0 = i1 ^ p2;
  *y1 = i2 ^ p3;
}
```

Insecure p2 and p3

After front-end optimizations by HLS

Impact of HLS on PSCA Security

Back-End HLS Optimizations

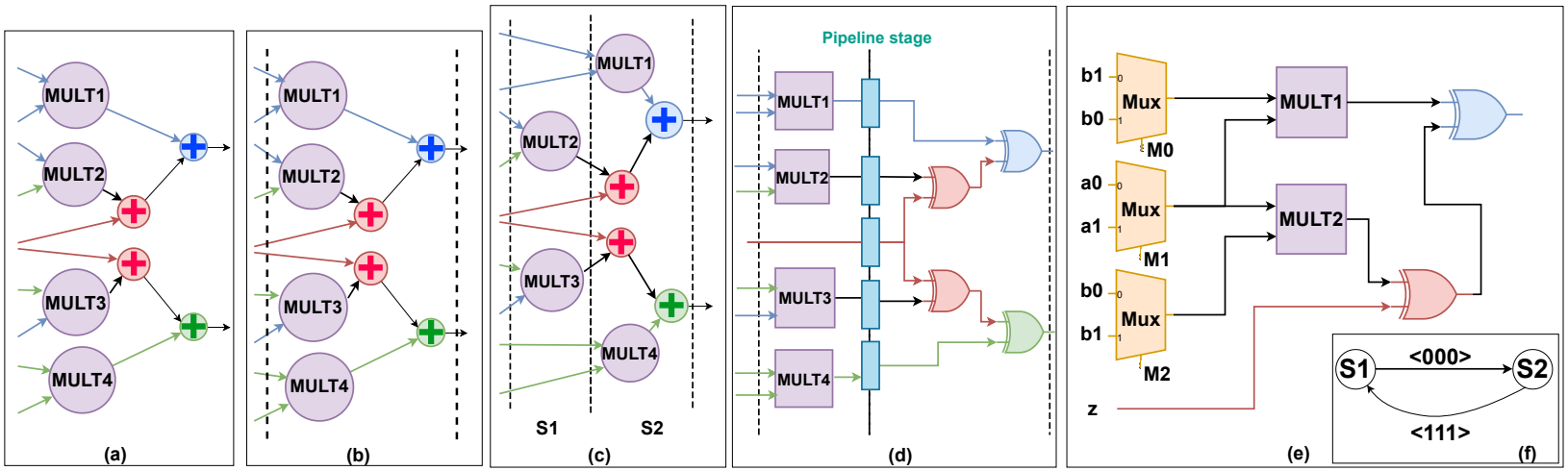


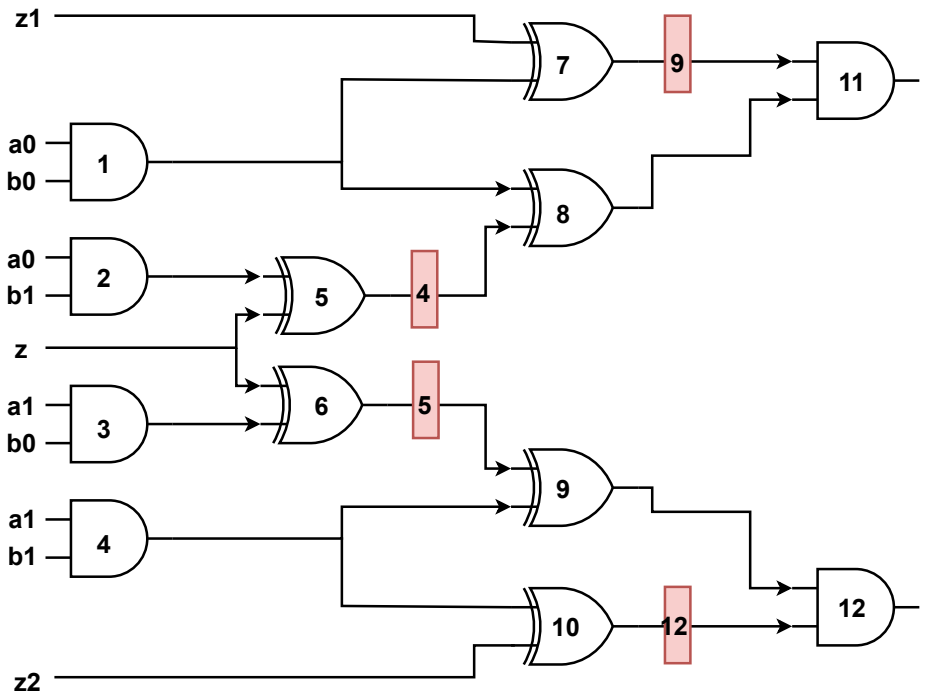
Figure 5: Impact of HLS optimization on PSCA security at various steps in the HLS flow

Domain-Specific HLS for Masking: What is really needed?

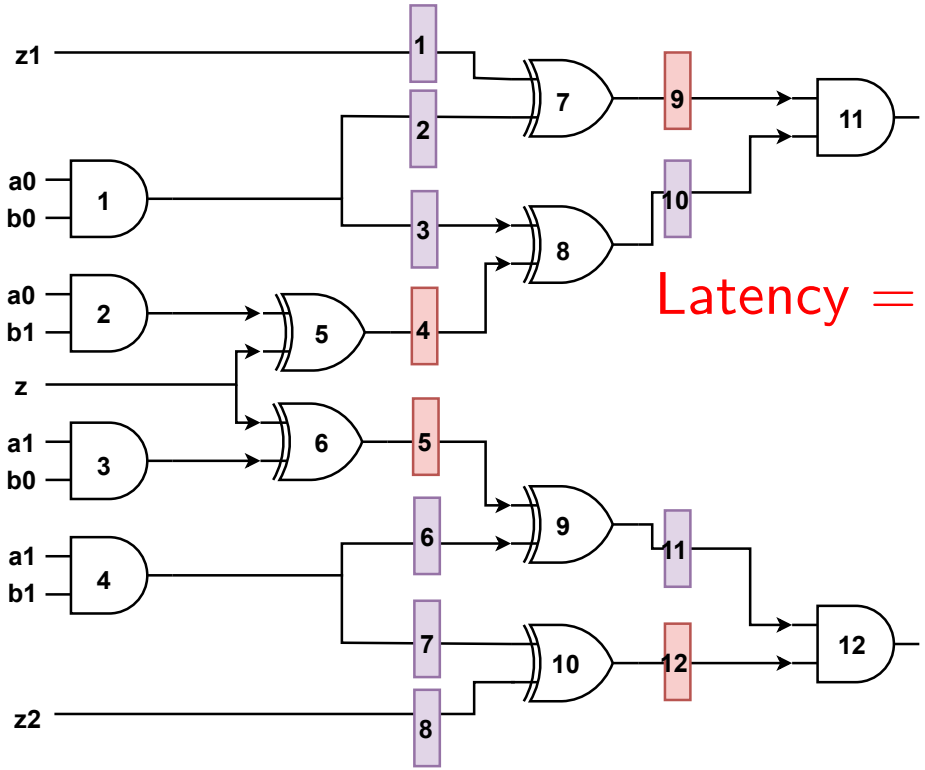
- No optimizations. MaskedHLS does no optimizations but a one-to-one translation to RTL
- Correct and efficient insertion of registers in the output according to input annotations.
  - Efficient? minimum latency
  - Correct? registers inserted at positions required by masking and balanced parallel paths with minimum latency

Inserting Registers at designated locations: Problem?

Unbalanced delay due to different number of registers before both inputs of a gate.



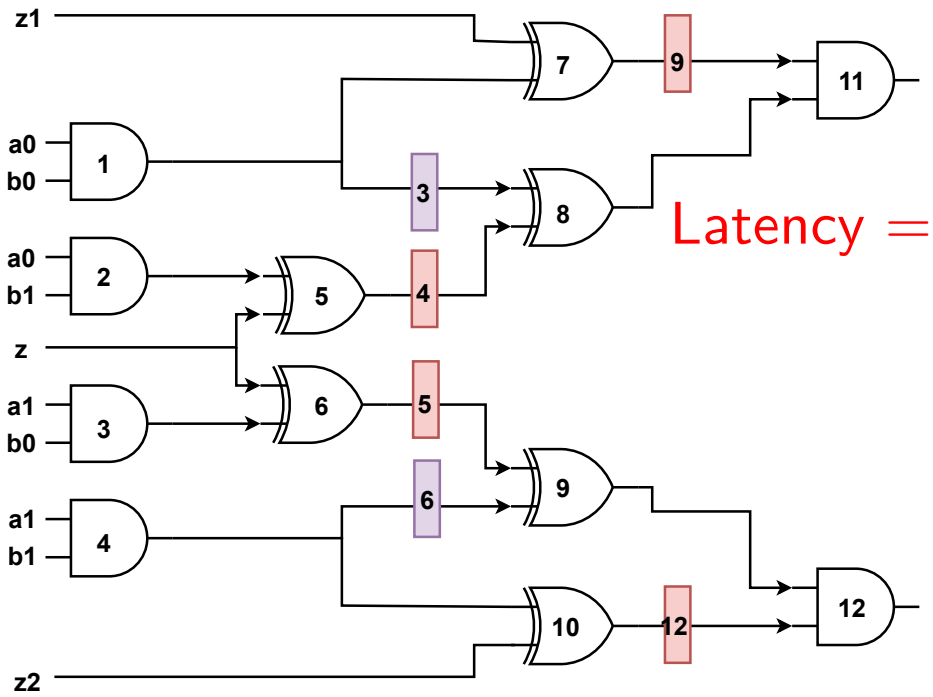
Inserting Registers at designated locations: Problem?



Latency = 2, Registers = 12

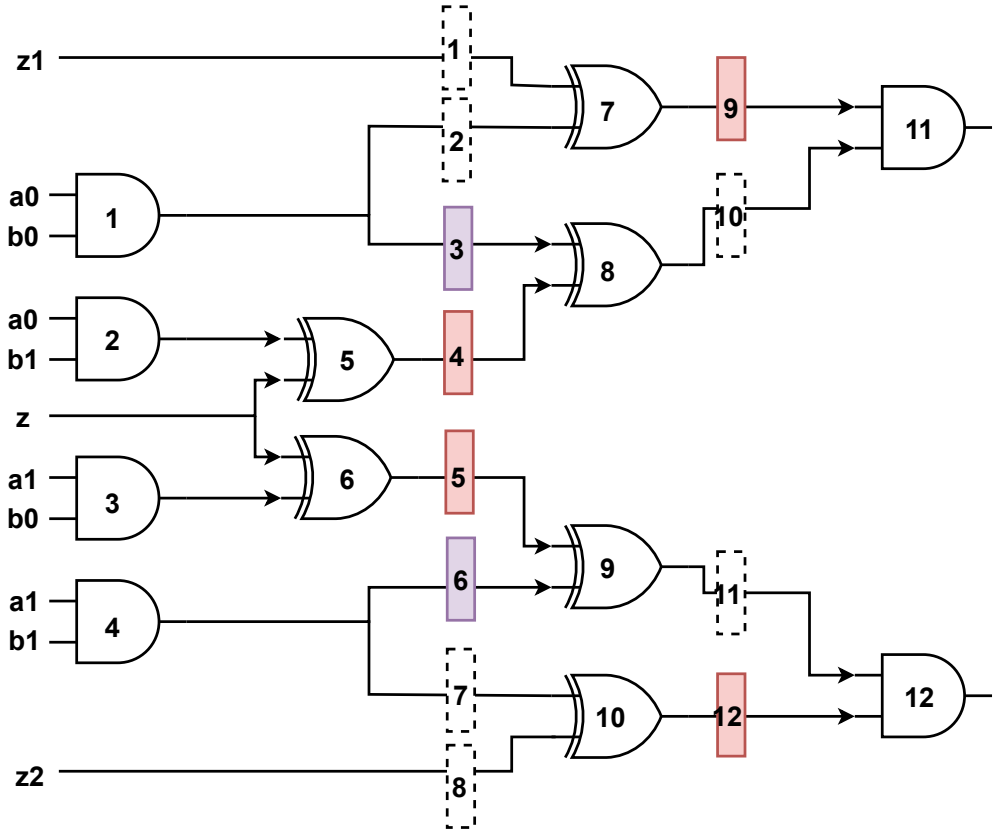
Can we reduce the latency of this circuit while **still having registers 4, 5, 9, and 12** and also **balanced parallel paths**?

Inserting Registers at designated locations: Problem?



Manually finding the optimal set of such balancing registers is difficult **specially for larger cryptographic designs like the AES S-box**

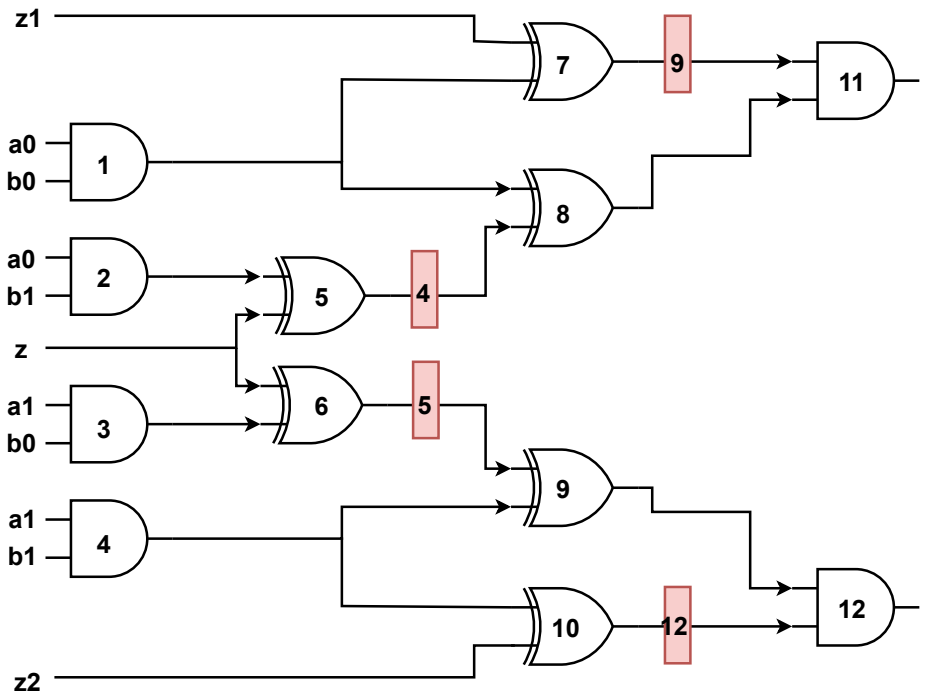
Register Balancing: How to do it automatically?



Have to look for automated ways to insert registers and balance them in parallel paths **with minimum latency**.

Register Balancing: Identifying minimum possible latency

We can do a Depth First Search (DFS) to find out the **Maximum number of registers in a path**.

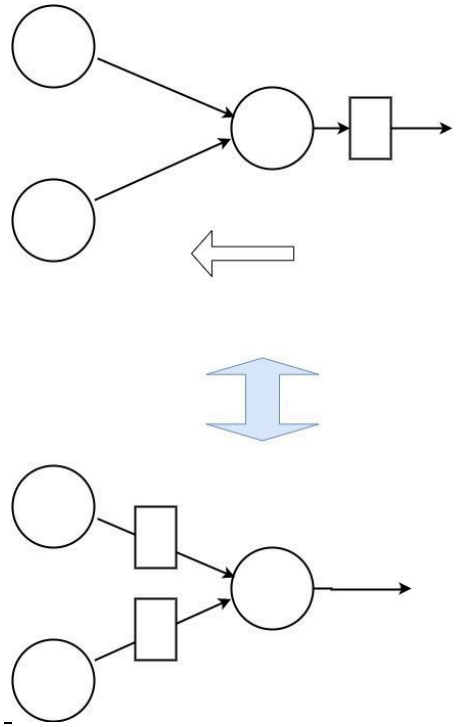


Maximum number of registers in a path = Minimum possble latency

Retiming

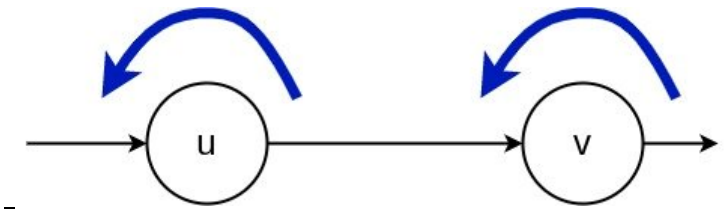
How to move registers across a circuit automatically?

Retiming Shifts registers within a circuit without altering functionality.



Retiming

Retiming label ($r(v)$): The number of registers moved from the outputs to the input of the vertex v. Therefore, retiming is defined as assigning retiming labels to all the design units of the circuit.



If the edge weights $w(e_{u,v})$ in the original circuit changes to an edge weight $w^r(e_{u,v})$ after retiming, then:

$$w^r(e_{u,v}) = r(v) - r(u) + w(e_{u,v})$$

Retiming Constraints

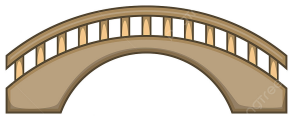
Given retimiing labels for each node in the design, we can enfoce certain constraints on the retiming labels for a correct circuit implementation.

- **Feasibility Constraints (for each edge)**: enforce that *the number of registers in an edge after moving registers using retiming is non-negative*.
- **Critical Path Constraints (for each path)**: enforce that *the computational delay of any path after moving registers using retiming should be less than the target clock period of the circuit*.

How to use the constraints for Register Balancing?

Can we modify the design so that the critical path constraints of an unbalanced design fails?

Constraints



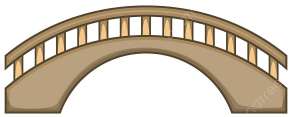
Balanced design

Solving these constraints for the retiming labels will then give us the **retiming labels for the balanced design**

How to use the constraints for Register Balancing?

Can we use the modification to ensure minimum latency of the design after it is retimed to satisfy the constraints

Min. latency



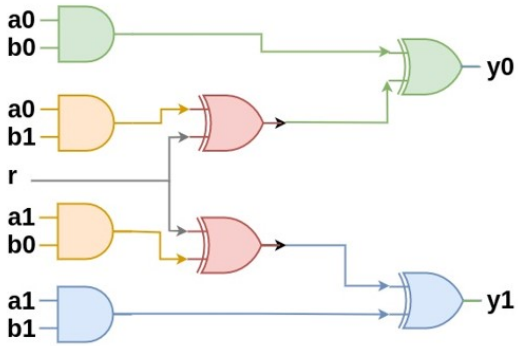
Balanced design

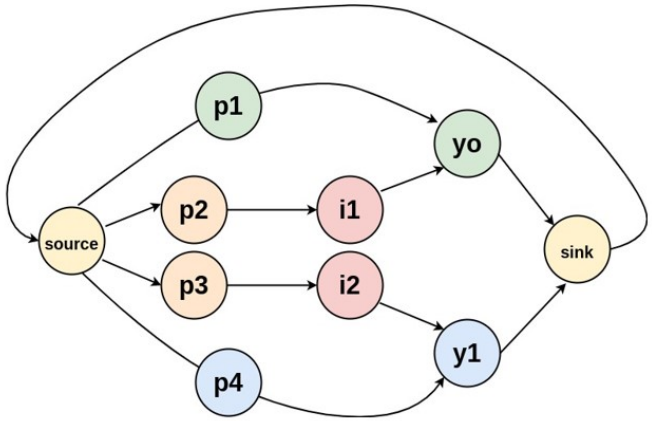
Constraints

If so then such a **modification** solution will also ensure **minimum latency**

HLS Model: the Bridge

Form the Abstract Syntax Tree (AST) of the design (at software) create a **Retiming model**:

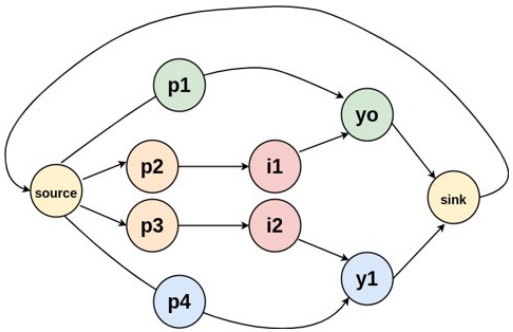


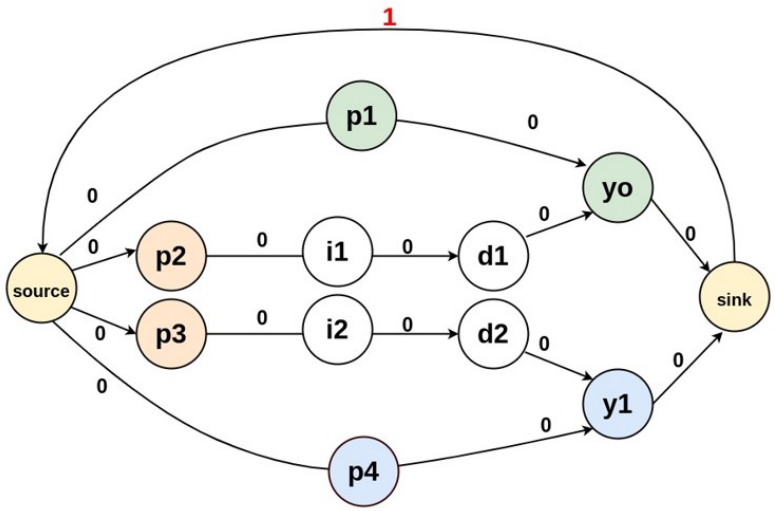


Add **source** and **sink**.

Add an edge from **sink** -> **source**

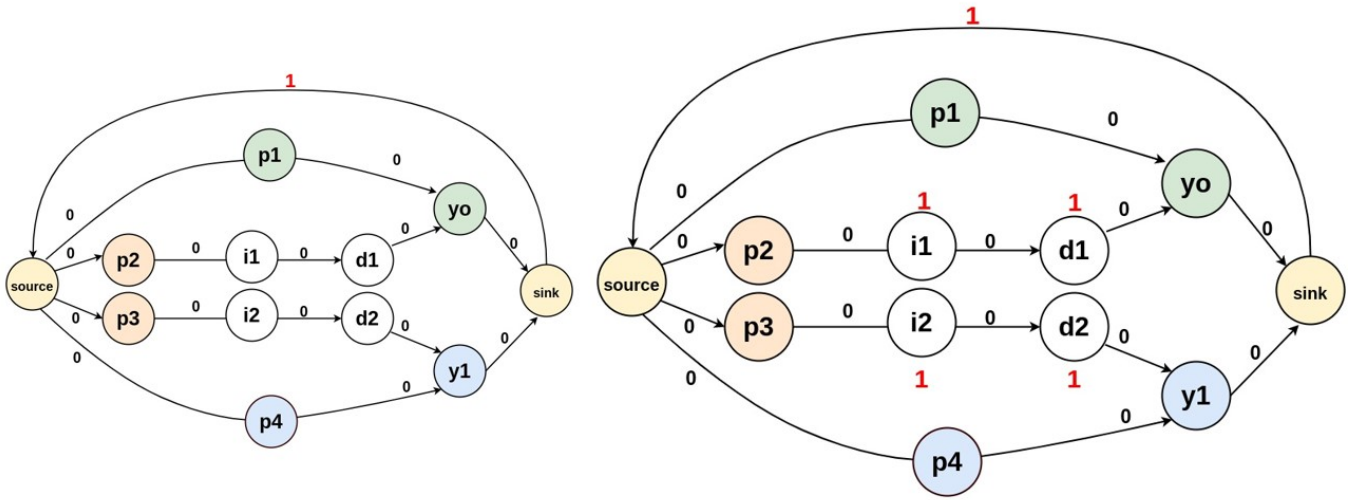
HLS Model: the Bridge





Add a **dummy node** after each node that requires a register insertion.
Weight(sink -> source) = minimum possible latency (determined by DFS)

HLS Model: the Bridge



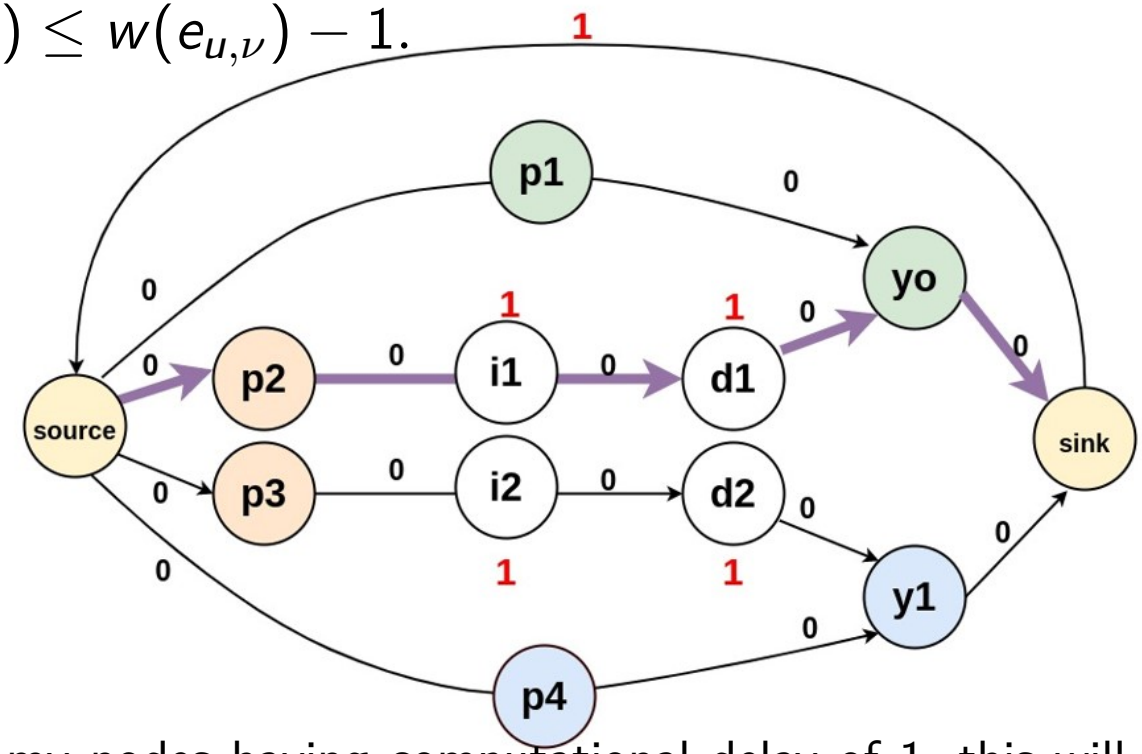
For each node after which a register needs to be inserted and for each dummy node **computational delay = 1**.

For rest of the nodes it is 0.

Critical Path Constraints on the Retiming Labels in this *HLS model*

Enforce that the computational delay of any path should be less than the target clock period c of the circuit.

$r(u) - r(\nu) \leq w(e_{u,\nu}) - 1.$

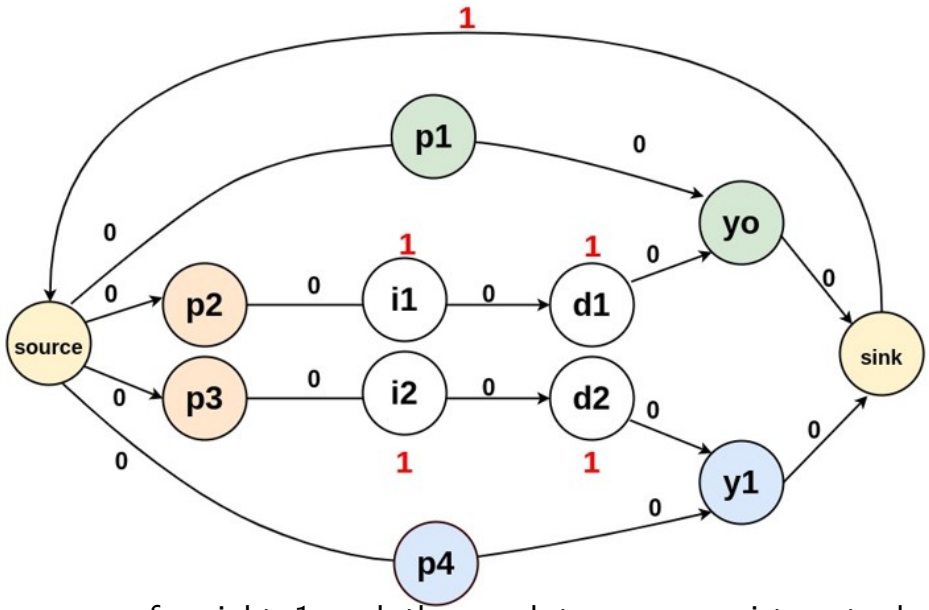


Due to the dummy nodes having computational delay of 1, this will force the Critical Path Constraints to be violated between the dummy node and the node requiring register insertion.

Feasibility Constraints on the Retiming Labels in this *HLS model*

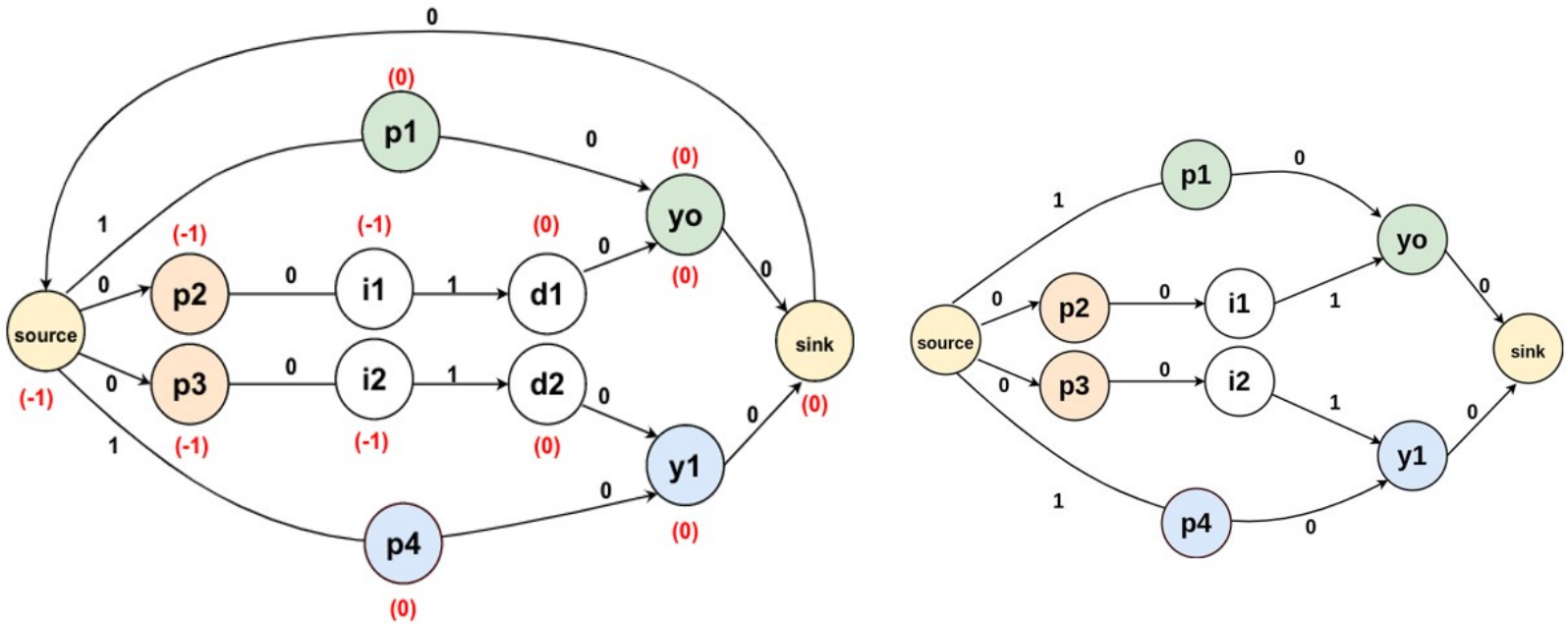
Enforce that the number of registers in an edge after retiming is non-negative.

$r(\nu) - r(u) \leq w(e_{u,\nu}), \quad$ for all $e_{u,\nu} \in E$.



Due to the edge *sink*$->$ *source* of weight 1 and the need to move registers to break the **critical path**, this will move 1 register from source to $i1$ and $i2$. Feasibility constraints will ensure that **equal number of registers equal to** 1 are moved across *source* in all outgoing paths, ensuring minimum latency and balanced parallel paths

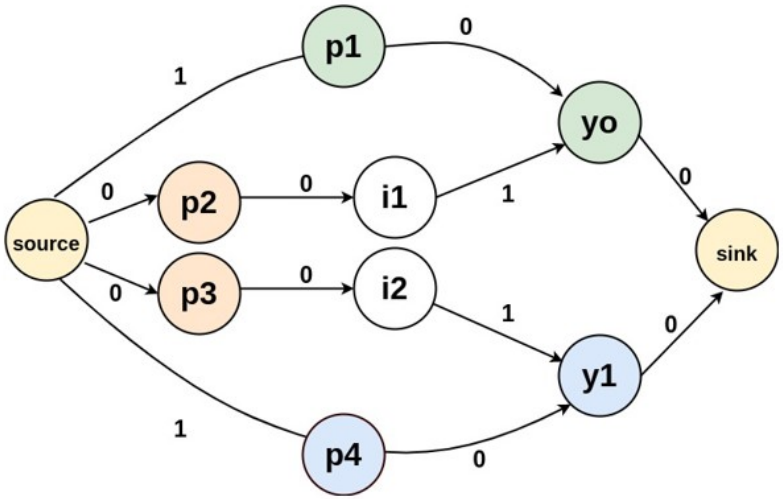
Solve the Constraints on the Retiming Labels

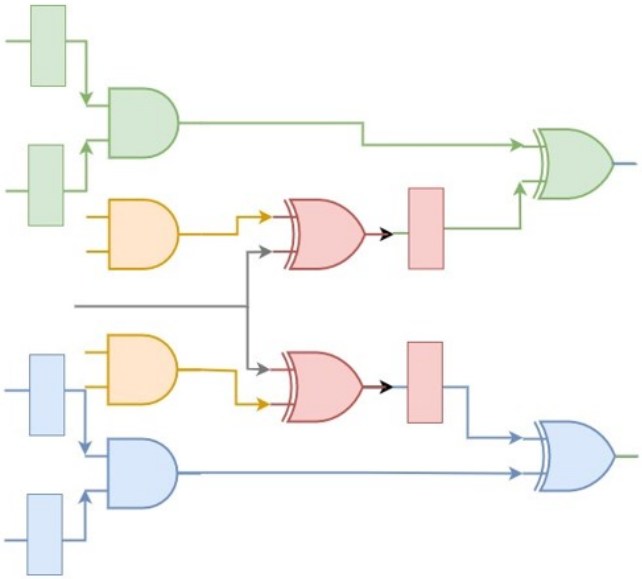


Thus solving such constraints will result in retiming labels such that registers are inserted at required locations (between the dummy and the node requiring insertion) and parallel paths are balanced.

Obtain Balanced *HLS model*

By virtue of retiming, all parallel paths will be balanced with equal number of registers.





Balanced Design

MaskedHLS Flow

- **Steps**:
  1. **Input:** shared masked software using gadgets.
  2. **Output:** masked hardware at RTL with registers inserted in designated locations and balanced parallel paths with minimum latency.
  3. Step 1: annotate input for register insertion.
  4. Step 2: parse the input and the annotations and construct the *HLS-model*.
  5. Step 3: insert registers and balance parallel paths using retiming-based approach.
  6. Step 4: one-to-one translation to RTL.
- **Focus**: Insertion and balancing of registers at RTL; Minimizing latency and registers while ensuring side-channel security.

MaskedHLS Flow

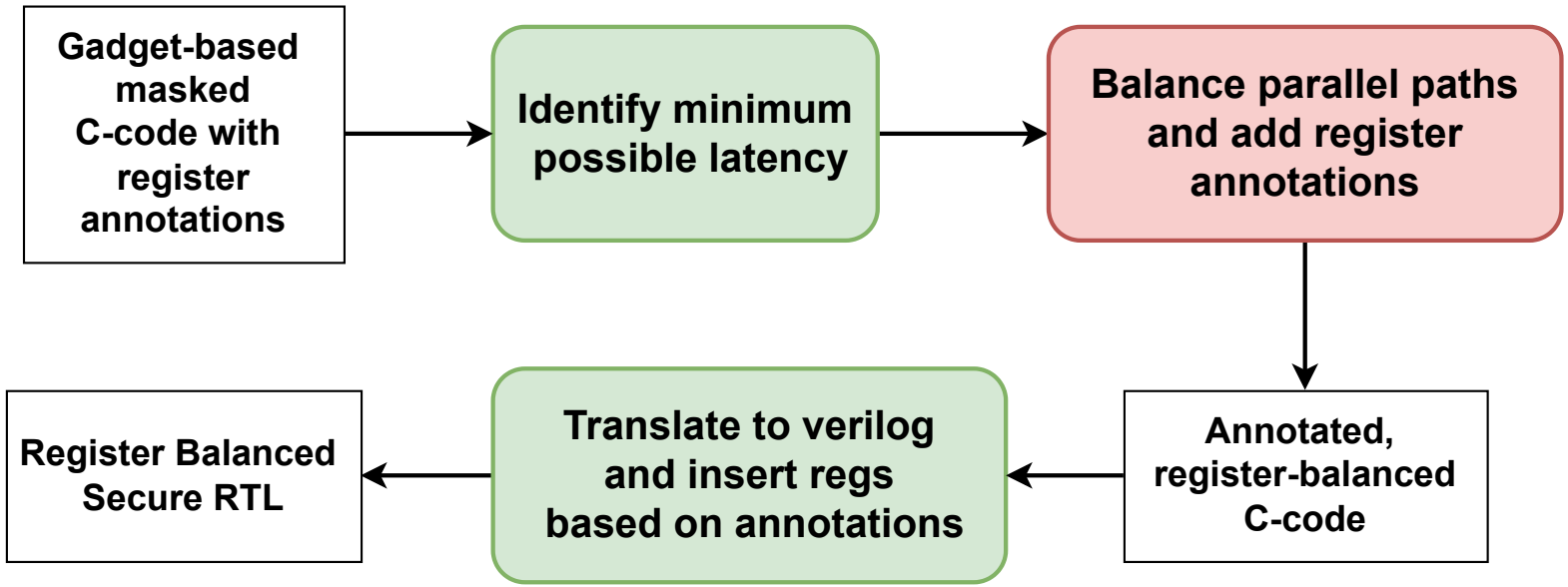


Figure 6: MaskedHLS flow

MaskedHLS Detailed Flow

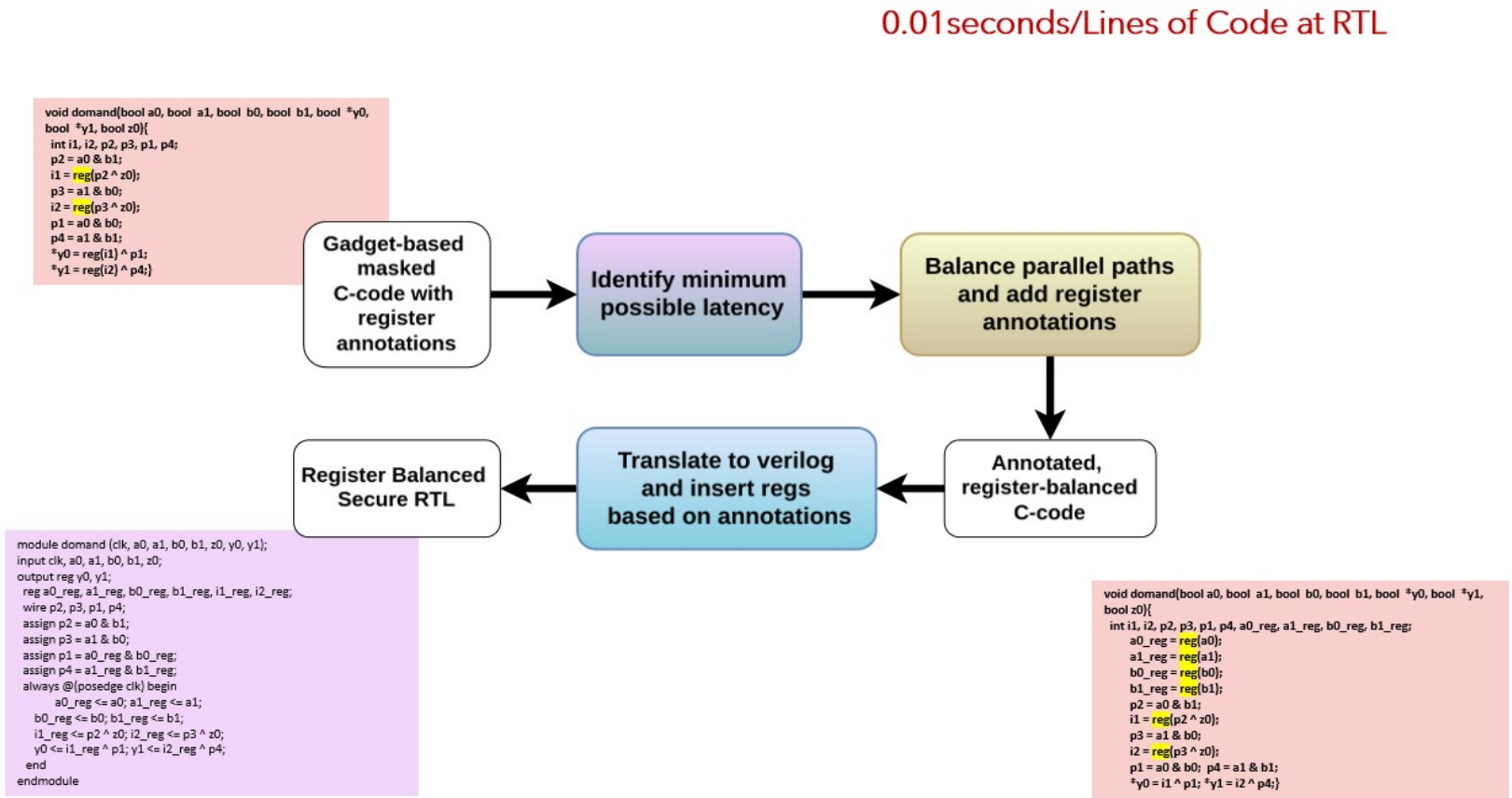


Figure 7: MaskedHLS Detailed flow

Outline

1 Background
- PSCAs
- Masking
- Masking Gadgets
- Hardware vs Software Masking
- Software Masking
- Hardware Masking
- HLS

2 MaskedHLS: domain-specific HLS
- Impact of HLS
- Domain-Specific HLS for Masked Designs
- MaskedHLS Flow

3 Results
- Results and Conclusion

Experimental Results

- **Benchmarks**:
  - PRESENT Cipher's S-box and AES Canrights S-box.
  - Tested with DOM, HPC1, HPC2, and COMAR gadgets.
- **Results**:
  - 73.9% register reduction.
  - 45.7% latency reduction.
  - PSCA security maintained across all designs.

Bogdanov, Andrey, et al. "PRESENT: An ultra-lightweight block cipher." Cryptographic Hardware and Embedded Systems-CHES 2007: 9th International Workshop, Vienna, Austria, September 10-13, 2007. Proceedings 9. Springer Berlin Heidelberg, 2007. Canright, David. "A very compact S-box for AES." International Workshop on Cryptographic Hardware and Embedded Systems. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005.

Synthesis Results

Table 1: Results for MaskedHLS

Design	#ann_regs	#bal_regs	#total_regs	#C	#nodes	#RTL	Runtime (s)
PRESENT_DOMAND	16	36	52	83	105	299	0.33
PRESENT_HPC1	32	68	100	84	169	454	0.40
PRESENT_HPC2	48	82	130	91	168	420	0.33
PRESENT_COMAR	56	38	94	94	209	515	0.88
AES_DOMAND	72	999	1071	485	1308	5307	11.22
AES_HPC1	216	1689	1905	515	1668	7707	25.77
AES_HPC2	432	1587	2019	481	1884	7875	64.87
AES_COMAR	468	2486	2954	495	2322	10261	119.17

Even for large cryptographic designs, the runtime of MaskedHLS is not high.

Overhead Results

Table 2: Area and Timing Overhead Comparison with Designs Without Registers

Design	Area (wo_reg)	Area (w_reg)	Timing (wo_reg)	Timing (w_reg)
PRESENT_unmasked	940.11	NA	1.11	NA
PRESENT_DOMAND	2639.22	5546.05	1.64	0.93
PRESENT_HPC1	2614.83	8815.18	1.69	0.83
PRESENT_HPC2	3220.92	10788.05	1.85	0.94
PRESENT_COMAR	2892.56	8936.05	1.82	0.98
PRESENT_average		**2.97x**		**0.52x**
AES_unmasked	55728.13	NA	18.99	NA
AES_DOMAND	1002202.72	1841877.19	28.01	5.72
AES_HPC1	1004612.10	3136636.19	28.94	4.12
AES_HPC2	1028632.47	2305685.39	31.60	4.50
AES_COMAR	134810.27	2727581.43	31.12	2.44
AES_average		**6.85x**		**0.13x**

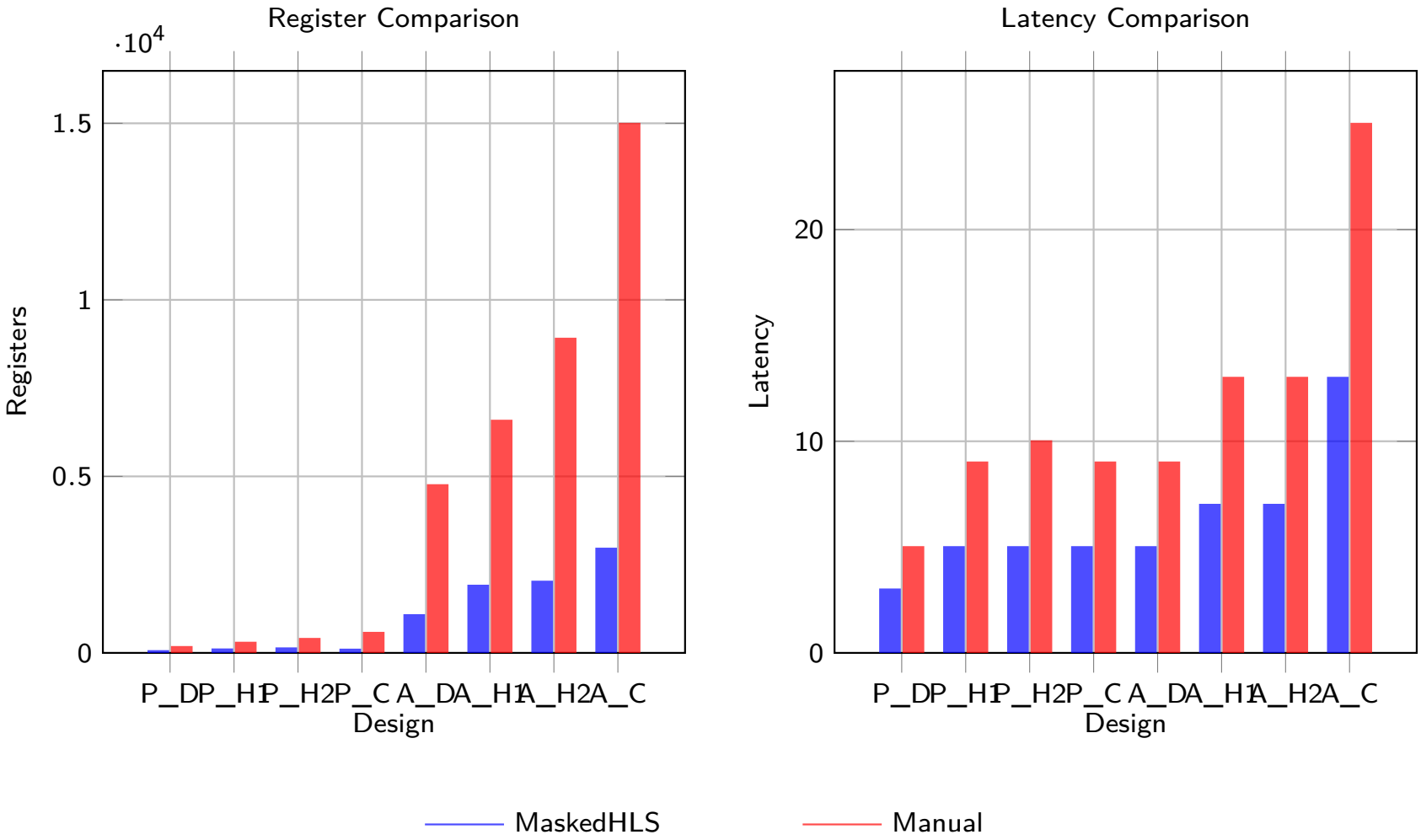
Note: wo_reg corresponds to the gadget-based masked circuit without registers, w_reg corresponds to the output of MaskedHLS. Timing is in nanoseconds.

MaskedHLS Savings Results

Table 3: Comparison of Register and Latency Savings Using MaskedHLS and Manual Methods

Design	Registers			Latency		
	MaskedHLS	Manual	Saving (%)	MaskedHLS	Manual	Saving (%)
PRESENT_DOMAND	52	168	**69.0%**	3	5	**40.0%**
PRESENT_HPC1	100	290	**65.5%**	5	9	**44.5%**
PRESENT_HPC2	130	398	**67.3%**	5	10	**50.0%**
PRESENT_COMAR	94	570	**83.5%**	5	9	**44.5%**
AES_DOMAND	1071	4752	**77.4%**	5	9	**44.5%**
AES_HPC1	1905	6578	**71.0%**	7	13	**46.1%**
AES_HPC2	2019	8901	**77.3%**	7	13	**46.1%**
AES_COMAR	2954	14987	**80.2%**	13	25	**50.0%**
Average			**73.9%**			**45.7%**

Comparison of Register and Latency Savings



Comparision of Register and Latency Savings Aggregated

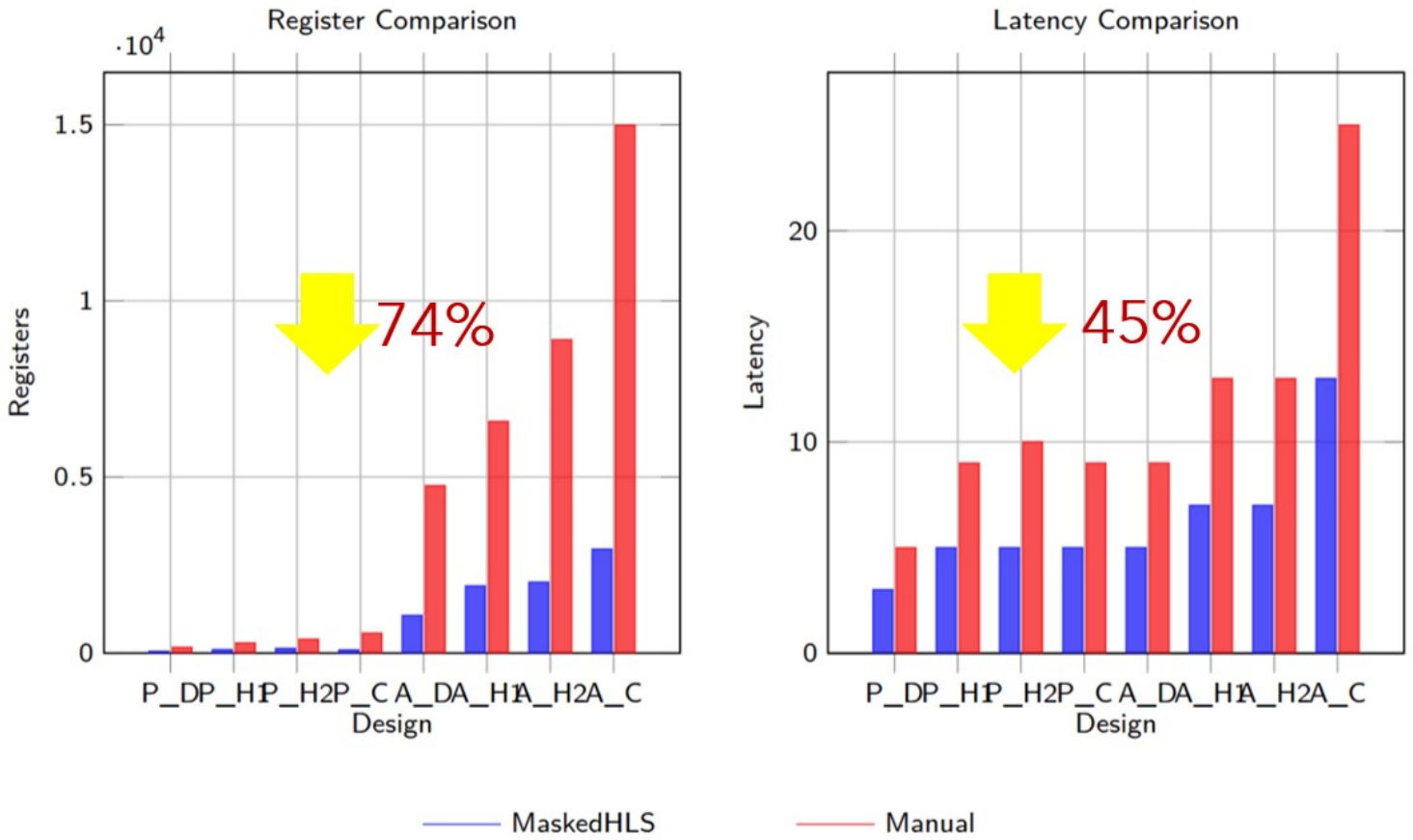


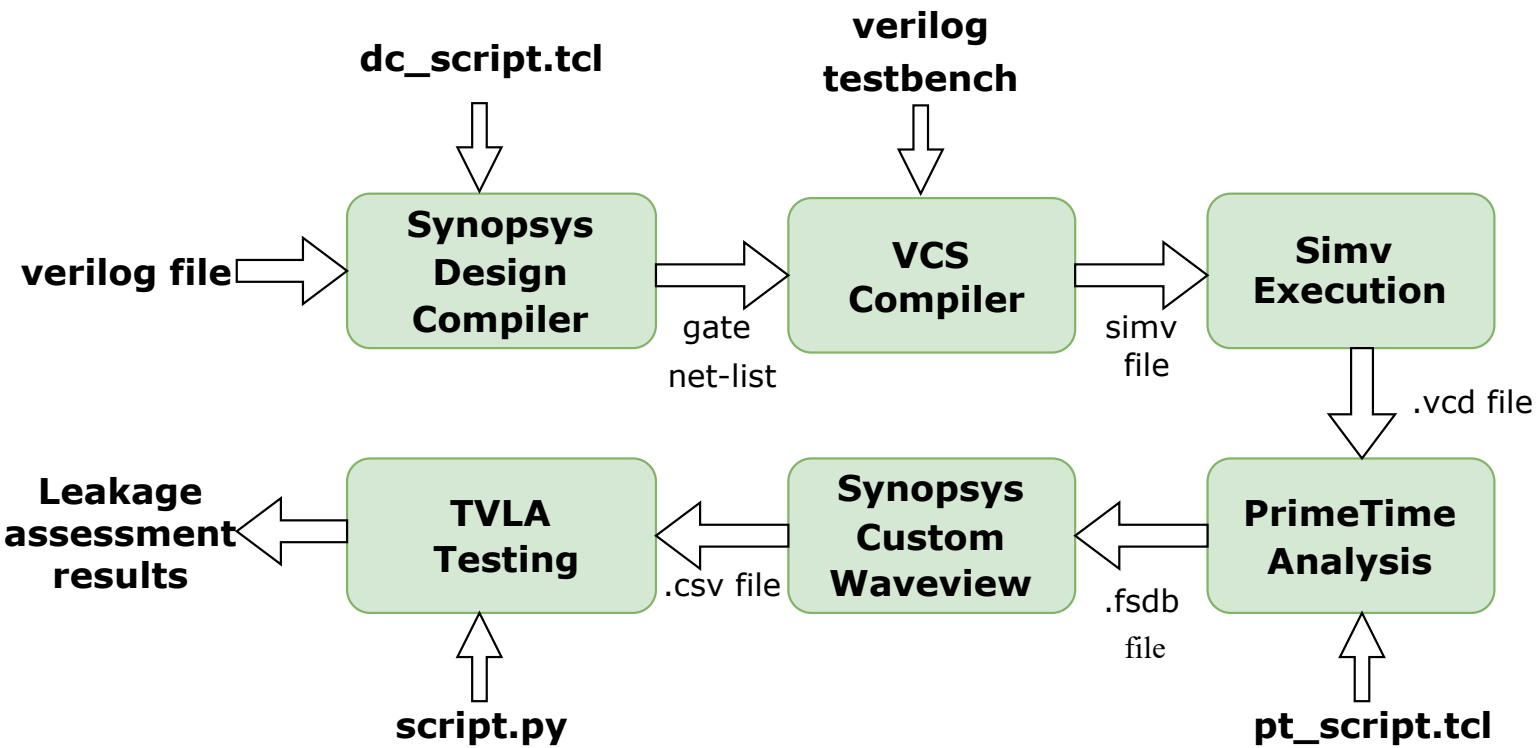
Figure 8: Savings of MaskedHLS versus Manual Register Balancing

TVLA Security Analysis

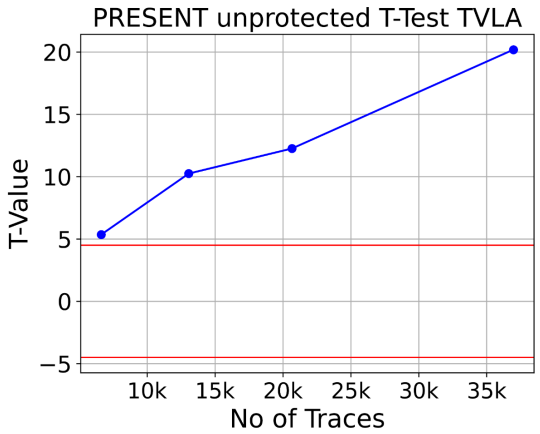
- **Test Vector Leakage Assessment (TVLA)**:
  - MaskedHLS designs showed significantly better security than unprotected designs.
  - AES COMAR design synthesized via MaskedHLS specially offers security till 100 million traces.

G. Becker et al., Test vector leakage assessment (TVLA) methodology in practice, in Proc. ICMC, 2013, p. 13.

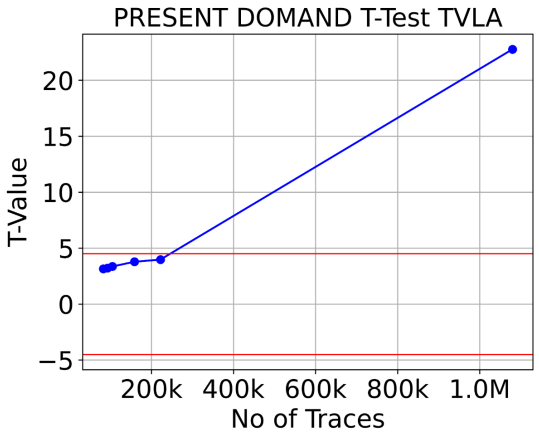
TVLA Security Analysis



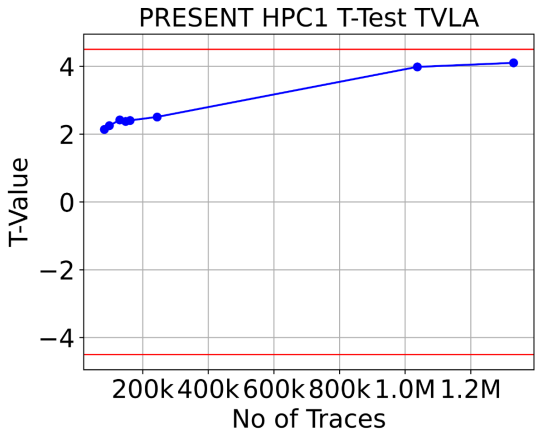
TVLA Results for PRESENT S-box



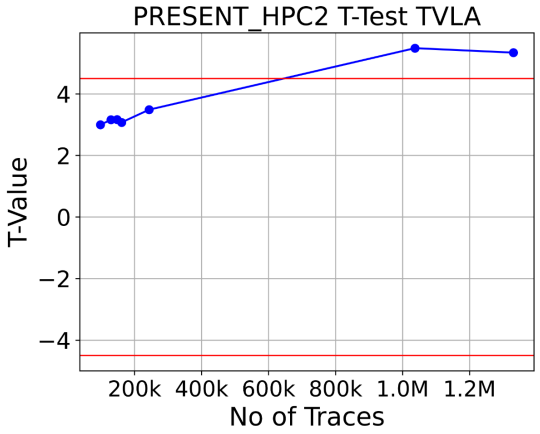
(a) PRESENT unprotected



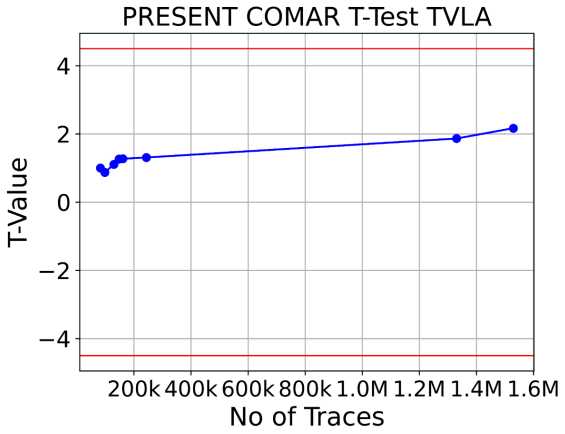
(b) PRESENT DOMAND



(c) PRESENT HPC1

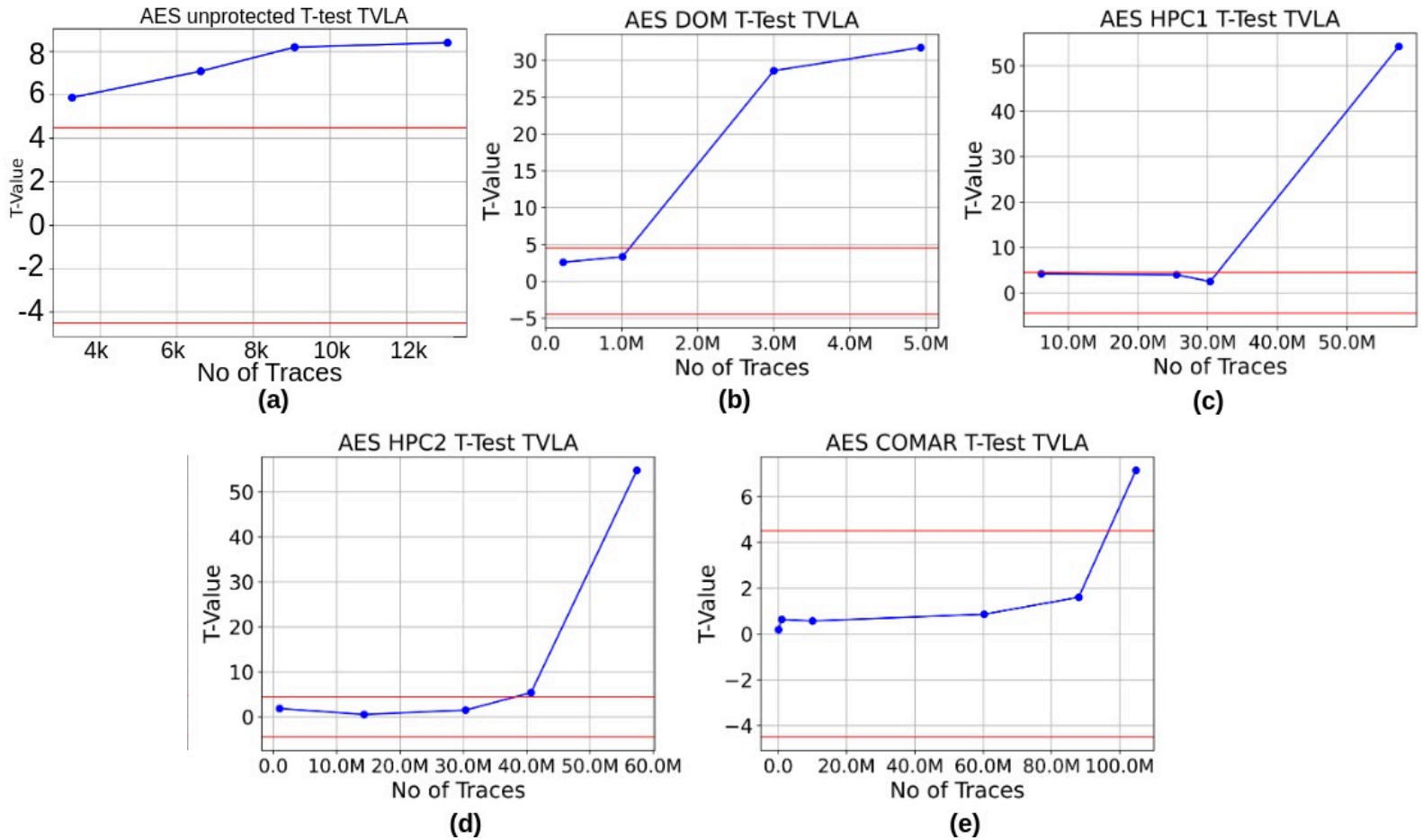


(d) PRESENT HPC2



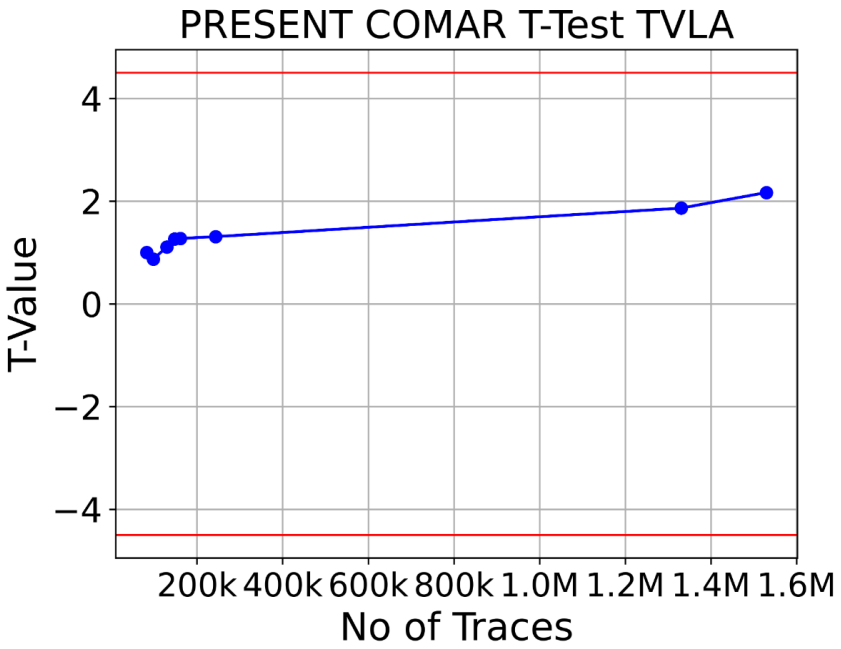
(e) PRESENT COMAR

TVLA Results for Canright's AES-256 S-box

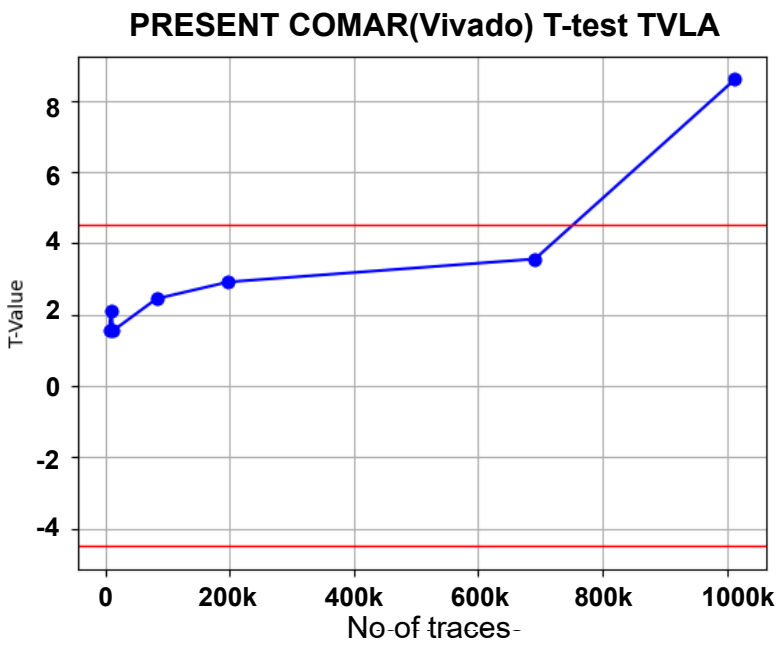


TVLA Comparison with VivadoHLS generated masked RTL

MaskedHLS generated



VivadoHLS generated



Conclusion

- **MaskedHLS**: An effective domain-specific HLS tool for generating PSCA-secure masked hardware.
- **Key Benefits**:
  - Significant register and latency reduction.
  - Robust security against power side-channel attacks.
- MaskedHLS is the first tool that automatically converts any masked cryptographic software into power-side channel secure masked RTL.
- Published in CASES 2024 (Best Paper Nomination)

Thanks

Thank you!
Questions?



Github